ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KĨ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP

Học sâu và ứng dụng trong thị giác máy tính

Phân loại ảnh MNIST bằng nhiều phương pháp

SVTH: Đàm Đức Huy (2670293)

TP. HỒ CHÍ MINH, 11/2026

Contents

1 Giới thiệu **1**
- 1.1 Bối cảnh và động lực . . . 1
- 1.2 Mục tiêu . . . 1
- 1.3 Các câu hỏi nghiên cứu . . . 2

2 Tập dữ liệu và thiết lập thực nghiệm **3**
- 2.1 Tập dữ liệu MNIST . . . 3
- 2.2 Tiền xử lý và phân chia dữ liệu . . . 3
- 2.3 Môi trường và cấu hình huấn luyện . . . 3
- 2.4 Quy trình huấn luyện và đánh giá . . . 4
- 2.5 Các tiêu chí đánh giá . . . 4

3 E1: Phân loại ảnh bằng Softmax, MLP và CNN **6**
- 3.1 Mục tiêu . . . 6
- 3.2 Phương pháp biểu diễn dữ liệu đầu vào . . . 6
  - 3.2.1 Biểu diễn vector cho Softmax và MLP . . . 6
  - 3.2.2 Biểu diễn không gian cho CNN . . . 7
- 3.3 Kiến trúc các mô hình . . . 7
  - 3.3.1 Softmax Classifier . . . 7
  - 3.3.2 MLP Classifier . . . 7
  - 3.3.3 CNN Classifier . . . 8
    - 3.3.3.1 Trích xuất đặc trưng . . . 8
    - 3.3.3.2 Phân loại đặc trưng . . . 9
- 3.4 Kết quả thực nghiệm . . . 9
  - 3.4.1 Nhận xét kết quả . . . 10
- 3.5 Phân tích lỗi và thảo luận . . . 10
  - 3.5.1 Softmax Classifier . . . 10
  - 3.5.2 MLP Classifier . . . 12
  - 3.5.3 CNN Classifier . . . 12
- 3.6 Kết luận E1 . . . 14

4 E2: Phân loại ảnh bằng Transformer và Self-Attention **15**
- 4.1 Mục tiêu . . . 15
- 4.2 Biểu diễn ảnh thành chuỗi token . . . 15
  - 4.2.1 Patch Embedding . . . 15
  - 4.2.2 Row Embedding . . . 16
  - 4.2.3 CNN Embedding . . . 16
  - 4.2.4 So sánh các phương pháp tokenization . . . 17
- 4.3 Cơ chế Multi-Head Self-Attention . . . 17
  - 4.3.1 Nguyên lý Self-Attention . . . 17

4.3.2 Multi-Head Self-Attention . . . 17
4.3.3 MSA tự xây dựng và MSA của PyTorch . . . 18
4.4 Kiến trúc Transformer Classifier . . . 18
4.5 Kết quả thực nghiệm . . . 19
4.5.1 Ảnh hưởng của phương pháp tokenization . . . 19
4.5.2 So sánh hai phiên bản MSA . . . 20
4.6 Phân tích lỗi và thảo luận . . . 20
4.6.1 Patch Embedding với Scratch MSA . . . 20
4.6.2 Row Embedding với Scratch MSA . . . 21
4.6.3 CNN Embedding với Scratch MSA . . . 21
4.6.4 Patch Embedding với PyTorch MSA . . . 24
4.7 Kết luận E2 . . . 24

5 E3: Phân loại ảnh bằng LSTM và GRU **26**
5.1 Mục tiêu . . . 26
5.2 Biểu diễn ảnh thành chuỗi . . . 26
5.2.1 Row Sequence . . . 26
5.2.2 Cơ chế xử lý tuần tự . . . 27
5.3 Kiến trúc LSTM . . . 27
5.3.1 Nguyên lý hoạt động . . . 27
5.3.2 Kiến trúc LSTM Classifier . . . 28
5.4 Kiến trúc GRU . . . 28
5.4.1 Nguyên lý hoạt động . . . 28
5.4.2 Kiến trúc GRU Classifier . . . 29
5.5 Kết quả thực nghiệm . . . 29
5.5.1 Nhận xét kết quả . . . 30
5.6 So sánh với các mô hình E1 . . . 30
5.7 Phân tích lỗi và thảo luận . . . 31
5.7.1 LSTM Classifier . . . 31
5.7.2 GRU Classifier . . . 32
5.7.3 Thảo luận chung . . . 32
5.8 Kết luận E3 . . . 34

6 Kết luận **35**

Danh sách hình vẽ

3.1 So sánh validation accuracy của Softmax, MLP và CNN qua các epoch 10
3.2 Confusion matrix của Softmax 11
3.3 Một số trường hợp dự đoán sai của Softmax 11
3.4 Confusion matrix của MLP 12
3.5 Một số trường hợp dự đoán sai của MLP 13
3.6 Confusion matrix của CNN 13
3.7 Một số trường hợp dự đoán sai của CNN 14

4.1 So sánh validation accuracy của các mô hình E2 19
4.2 Confusion matrix của Patch + Scratch 20
4.3 Các trường hợp dự đoán sai của Patch + Scratch 21
4.4 Confusion matrix của Row + Scratch 22
4.5 Các trường hợp dự đoán sai của Row + Scratch 22
4.6 Confusion matrix của CNN + Scratch 23
4.7 Các trường hợp dự đoán sai của CNN + Scratch 23
4.8 Confusion matrix của Patch + PyTorch 24
4.9 Các trường hợp dự đoán sai của Patch + PyTorch 25

5.1 So sánh validation accuracy của LSTM và GRU qua các epoch 30
5.2 Confusion matrix của LSTM 31
5.3 Một số trường hợp dự đoán sai của LSTM 32
5.4 Confusion matrix của GRU 33
5.5 Một số trường hợp dự đoán sai của GRU 33

Danh sách bảng

2.1 Phân chia tập dữ liệu MNIST 3
2.2 Cấu hình thực nghiệm chung 4

3.1 Kết quả thực nghiệm E1 9

4.1 So sánh ba phương pháp tokenization 17
4.2 Kết quả thực nghiệm E2 19

5.1 Kết quả thực nghiệm E3 30
5.2 So sánh LSTM và GRU với baseline E1 31

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực

Phân loại ảnh là một trong những bài toán cơ bản của thị giác máy tính, với mục tiêu xác định nhãn tương ứng cho một ảnh đầu vào dựa trên các lớp đã được định nghĩa trước. Sự phát triển của học sâu đã mang đến nhiều kiến trúc mạng nơ-ron có khả năng tự động học và trích xuất các đặc trưng từ dữ liệu ảnh, qua đó nâng cao hiệu quả của các hệ thống nhận dạng và phân loại.

Các kiến trúc học sâu tiếp cận bài toán phân loại ảnh theo nhiều hướng khác nhau, từ những mô hình cơ bản như Multi-Layer Perceptron (MLP) và Convolutional Neural Network (CNN), đến việc ứng dụng các kiến trúc vốn được sử dụng rộng rãi trong xử lý ngôn ngữ tự nhiên như Transformer, Long Short-Term Memory (LSTM) và Gated Recurrent Unit (GRU) vào dữ liệu hình ảnh. Sự đa dạng trong cách biểu diễn và xử lý dữ liệu đặt ra nhu cầu nghiên cứu, so sánh các kiến trúc nhằm đánh giá khả năng học đặc trưng, hiệu quả phân loại và chi phí tính toán của từng phương pháp.

1.2 Mục tiêu

Báo cáo này tập trung triển khai và đánh giá các mô hình học sâu thông qua ba bài tập E1, E2 và E3 thuộc môn học CO5085 – Học sâu và ứng dụng trong thị giác máy tính.

Các mục tiêu cụ thể bao gồm:

- **E1 – Phân loại ảnh bằng Softmax, MLP và CNN:**

  Xây dựng và huấn luyện ba mô hình phân loại ảnh, bao gồm Softmax Classifier,MLP và CNN.

  Thực hiện so sánh các mô hình dựa trên độ chính xác phân loại, số lượng tham số, quá trình hội tụ trong huấn luyện và các trường hợp dự đoán sai.

- **E2 – Transformer và cơ chế Multi-Head Self-Attention:**

  Hiện thực cơ chế Multi-Head Self-Attention (MSA) từ các phép toán cơ bản và xây dựng phiên bản sử dụng các module có sẵn của PyTorch.

  Nghiên cứu các phương pháp biểu diễn ảnh thành chuỗi token, bao gồm Patch Embedding, Row Embedding và CNN-based Embedding.

  Đánh giá ảnh hưởng của phương pháp tokenization và cách hiện thực MSA đến độ chính xác phân loại, số lượng tham số và thời gian huấn luyện.

- **E3 – Phân loại ảnh bằng LSTM và GRU:**

  Xây dựng các mô hình LSTM và GRU để thực hiện phân loại ảnh thông qua cách biểu diễn ảnh thành chuỗi.

  Trong đó, mỗi hàng của ảnh được xem như một bước thời gian, cho phép mô hình học và tích lũy thông tin theo thứ tự tuần tự.

  So sánh hiệu quả của LSTM và GRU dựa trên độ chính xác, số lượng tham số và khả năng học phụ thuộc theo chuỗi, đồng thời đối chiếu kết quả với các mô hình MLP và CNN trong bài tập E1.

1.3 Các câu hỏi nghiên cứu

Thông qua quá trình triển khai và thực nghiệm, báo cáo hướng đến việc trả lời các câu hỏi nghiên cứu sau:

1. Các kiến trúc Softmax Classifier, MLP và CNN có sự khác biệt như thế nào về độ chính xác phân loại, số lượng tham số và khả năng học đặc trưng trên cùng một tập dữ liệu ảnh?
2. Các phương pháp tokenization khác nhau, bao gồm Patch Embedding, Row Embedding và CNN-based Embedding, ảnh hưởng như thế nào đến hiệu quả phân loại của mô hình Transformer?
3. Cơ chế Multi-Head Self-Attention được hiện thực thủ công có sự khác biệt như thế nào về hiệu quả phân loại và chi phí tính toán so với phiên bản sử dụng module có sẵn trong PyTorch?
4. Các mô hình LSTM và GRU có thể khai thác thông tin tuần tự từ ảnh để thực hiện phân loại hiệu quả đến mức nào?
5. Những sự đánh đổi nào tồn tại giữa độ chính xác phân loại, độ phức tạp của mô hình và thời gian huấn luyện khi so sánh các kiến trúc được triển khai trong ba bài tập?

Chương 2

Tập dữ liệu và thiết lập thực nghiệm

2.1 Tập dữ liệu MNIST

Trong cả ba bài tập E1, E2 và E3, nhóm sử dụng tập dữ liệu MNIST để huấn luyện và đánh giá các mô hình phân loại ảnh.

MNIST là tập dữ liệu chữ số viết tay, bao gồm 70.000 ảnh xám thuộc 10 lớp tương ứng với các chữ số từ 0 đến 9. Mỗi ảnh có kích thước 28×28 pixel và một kênh màu.

Tập dữ liệu có 60.000 ảnh huấn luyện và 10.000 ảnh kiểm thử. Việc sử dụng cùng một tập dữ liệu giúp so sánh hiệu quả của các kiến trúc trong cùng điều kiện bài toán.

2.2 Tiền xử lý và phân chia dữ liệu

Các ảnh đầu vào được chuyển thành tensor thông qua `transforms.ToTensor()`, đồng thời chuyển giá trị pixel từ khoảng $[0, 255]$ về $[0, 1]$.

Tập huấn luyện ban đầu được chia ngẫu nhiên thành 50.000 ảnh dùng để huấn luyện và 10.000 ảnh dùng để xác thực. Việc phân chia sử dụng `random_split` với seed cố định bằng 42 để đảm bảo khả năng tái lập.

Tập kiểm thử gồm 10.000 ảnh được giữ riêng và chỉ sử dụng để đánh giá mô hình sau huấn luyện.

Bảng 2.1: *Phân chia tập dữ liệu MNIST*

Tập dữ liệu	Số lượng ảnh	Tỷ lệ
Training	50.000	71,43%
Validation	10.000	14,29%
Testing	10.000	14,29%
Tổng cộng	70.000	100%

Dữ liệu được đưa vào mô hình thông qua `DataLoader` với batch size bằng 128. Tập training được xáo trộn sau mỗi epoch, trong khi validation và test giữ nguyên thứ tự dữ liệu.

2.3 Môi trường và cấu hình huấn luyện

Các thí nghiệm được thực hiện trên Kaggle Notebook, sử dụng Python và framework PyTorch với khả năng tăng tốc bằng GPU.

Môi trường thực nghiệm và các siêu tham số huấn luyện được tổng hợp trong Bảng 2.2.

Bảng 2.2: *Cấu hình thực nghiệm chung*

Thành phần	Cấu hình
Nền tảng	Kaggle Notebook
Framework	PyTorch 2.11.0
Torchvision	0.26.0
GPU	NVIDIA Tesla T4
Batch size	128
Epochs	10
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Random seed	42

Các mô hình được huấn luyện với cùng cấu hình cơ bản nhằm tạo điều kiện thuận lợi cho việc so sánh kết quả. Những khác biệt về kiến trúc và cách biểu diễn dữ liệu sẽ được trình bày trong từng chương tương ứng.

2.4 Quy trình huấn luyện và đánh giá

Quá trình huấn luyện được xây dựng trực tiếp bằng PyTorch và sử dụng chung cho các mô hình.

Tại mỗi batch, dữ liệu được truyền qua mô hình để tạo logits. Giá trị loss được tính bằng CrossEntropyLoss, sau đó thuật toán backpropagation được sử dụng để tính gradient. Optimizer Adam cập nhật các tham số của mô hình dựa trên gradient thu được.

Sau mỗi epoch, mô hình được đánh giá trên tập validation mà không cập nhật trọng số. Các giá trị training loss, validation loss, training accuracy và validation accuracy được lưu lại để theo dõi quá trình hội tụ.

Checkpoint có validation loss thấp nhất được lưu và sử dụng để đánh giá trên tập test sau khi kết thúc quá trình huấn luyện.

Thời gian huấn luyện được đo bằng thời gian thực thi của hàm `fit()`, bao gồm quá trình huấn luyện và đánh giá validation qua các epoch. Chỉ số này được sử dụng để so sánh chi phí thực thi giữa các mô hình.

2.5 Các tiêu chí đánh giá

Hiệu quả của các mô hình được đánh giá thông qua các tiêu chí sau:

- **Accuracy:** Tỷ lệ mẫu được dự đoán đúng trên tổng số mẫu được đánh giá, được xác định bởi:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}} \times 100\% \tag{2.1}$$

- **Loss:**

  CrossEntropyLoss được sử dụng để đo lường mức độ sai lệch giữa phân phối xác suất dự đoán của mô hình và nhãn thực tế. Hàm mất mát được xác định như sau:

$$\mathscr{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\log\left(\frac{e^{z_{i,y_i}}}{\sum_{j=1}^{C}e^{z_{i,j}}}\right) \tag{2.2}$$

Trong đó:

- N: Số lượng mẫu trong batch.
- C: Số lớp phân loại ($C = 10$ với MNIST).
- $z_{i,j}$: Logit của mẫu thứ i đối với lớp j.
- y_i: Nhãn thực tế của mẫu thứ i.
- z_{i,y_i}: Logit tương ứng với lớp đúng.

Hàm CrossEntropyLoss kết hợp phép biến đổi LogSoftmax và Negative Log-Likelihood, do đó mô hình chỉ cần trả về logits mà không cần áp dụng Softmax trước khi tính loss.

Giá trị loss càng nhỏ cho thấy mô hình càng gán xác suất cao cho các nhãn đúng. Đường cong training loss và validation loss được sử dụng để theo dõi quá trình hội tụ và phát hiện dấu hiệu overfitting.

- **Số lượng tham số:** Tổng số tham số có thể học của mô hình, được sử dụng để đánh giá quy mô và độ phức tạp của từng kiến trúc.
- **Thời gian huấn luyện:** Tổng thời gian thực thi quá trình huấn luyện, tính bằng giây.
- **Phân tích lỗi:** Confusion matrix và các mẫu phân loại sai được sử dụng để phân tích những trường hợp mô hình gặp khó khăn trong quá trình dự đoán.

Các kết quả thực nghiệm và phân tích chi tiết theo từng tiêu chí được trình bày trong các chương E1, E2 và E3.

Chương 3

E1: Phân loại ảnh bằng Softmax, MLP và CNN

3.1 Mục tiêu

Bài tập E1 tập trung xây dựng và so sánh ba kiến trúc phân loại ảnh gồm Softmax Classifier, MLP và CNN trên tập dữ liệu MNIST.

Các mô hình được thiết kế với mức độ phức tạp tăng dần, từ phép biến đổi tuyến tính, mạng nhiều lớp kết nối đầy đủ đến mạng tích chập khai thác cấu trúc không gian của ảnh.

Quá trình thực nghiệm nhằm đánh giá ảnh hưởng của kiến trúc và phương pháp biểu diễn dữ liệu đến độ chính xác phân loại, số lượng tham số, thời gian huấn luyện và khả năng hội tụ của mô hình.

3.2 Phương pháp biểu diễn dữ liệu đầu vào

Mỗi ảnh MNIST có kích thước 28×28 pixel với một kênh màu. Tùy theo kiến trúc, dữ liệu đầu vào được xử lý theo hai cách khác nhau.

3.2.1 Biểu diễn vector cho Softmax và MLP

Softmax Classifier và MLP sử dụng phép Flatten để chuyển ảnh hai chiều thành vector một chiều.

Với ảnh đầu vào $X \in \mathbb{R}^{1\times 28\times 28}$, phép biến đổi được biểu diễn như sau:

$$X \longrightarrow x \in \mathbb{R}^{784} \tag{3.1}$$

Trong đó:

$$784 = 28 \times 28 \tag{3.2}$$

Phép Flatten giữ nguyên toàn bộ giá trị pixel nhưng thay đổi cách tổ chức dữ liệu từ ma trận hai chiều thành vector.

Do đó, các lớp kết nối đầy đủ không trực tiếp khai thác quan hệ không gian giữa các pixel lân cận thông qua cấu trúc hai chiều của ảnh.

3.2.2 Biểu diễn không gian cho CNN

Khác với Softmax và MLP, CNN tiếp nhận ảnh dưới dạng tensor ba chiều gồm số kênh, chiều cao và chiều rộng:

$$X \in \mathbb{R}^{1\times 28\times 28} \tag{3.3}$$

Các lớp Convolution sử dụng những kernel có kích thước nhỏ để trích xuất đặc trưng tại các vùng cục bộ của ảnh.

Nhờ đó, CNN có thể học những đặc trưng như cạnh, nét cong và các cấu trúc hình dạng, đồng thời duy trì thông tin không gian trong quá trình trích xuất đặc trưng.

Sau các lớp Convolution và Pooling, các feature map mới được Flatten thành vector để đưa vào phần phân loại cuối cùng.

3.3 Kiến trúc các mô hình

3.3.1 Softmax Classifier

Softmax Classifier là mô hình đơn giản nhất được sử dụng trong bài tập E1.

Sau khi ảnh được Flatten thành vector 784 phần tử, mô hình sử dụng một lớp Linear để ánh xạ dữ liệu đầu vào thành 10 giá trị logits, tương ứng với 10 lớp chữ số.

Phép biến đổi tuyến tính được mô tả bởi:

$$z = Wx + b \tag{3.4}$$

Trong đó x là vector đầu vào, $W \in \mathbb{R}^{10\times 784}$ là ma trận trọng số, $b \in \mathbb{R}^{10}$ là vector bias và $z \in \mathbb{R}^{10}$ là đầu ra logits.

Kiến trúc Softmax Classifier gồm:

$$\text{Flatten} \rightarrow \text{Linear}(784, 10) \rightarrow \text{Logits}$$

Tổng số tham số của mô hình là:

$$P_{\text{Softmax}} = 784 \times 10 + 10 = 7\,850 \tag{3.5}$$

Do chỉ thực hiện biến đổi tuyến tính, mô hình có độ phức tạp thấp nhưng khả năng biểu diễn các quan hệ phi tuyến trong dữ liệu còn hạn chế.

3.3.2 MLP Classifier

MLP mở rộng Softmax Classifier bằng cách sử dụng nhiều lớp kết nối đầy đủ kết hợp với các hàm kích hoạt phi tuyến.

Trong thực nghiệm, ảnh đầu vào tiếp tục được Flatten thành vector 784 phần tử, sau đó lần lượt đi qua hai lớp ẩn có 128 và 64 neuron.

Kiến trúc mô hình được biểu diễn như sau:

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

Giữa các lớp Linear, hàm kích hoạt ReLU được sử dụng để bổ sung khả năng biểu diễn các quan hệ phi tuyến:

$$\text{ReLU}(x) = \max(0, x) \tag{3.6}$$

Các phép biến đổi của mô hình được biểu diễn:

$$h_1 = \text{ReLU}(W_1 x + b_1) \tag{3.7}$$
$$h_2 = \text{ReLU}(W_2 h_1 + b_2) \tag{3.8}$$
$$z = W_3 h_2 + b_3 \tag{3.9}$$

Trong đó h_1 và h_2 là các biểu diễn đặc trưng tại hai lớp ẩn, còn z là vector gồm 10 logits.
Đầu ra được chuyển thành xác suất bằng Softmax khi cần thực hiện dự đoán.
Số lượng tham số của mô hình được tính:

$$\begin{aligned} P_{\text{MLP}} &= (784 \times 128 + 128) \\ &\quad + (128 \times 64 + 64) \\ &\quad + (64 \times 10 + 10) \qquad (3.10) \\ &= 109\,386 \qquad (3.11) \end{aligned}$$

So với Softmax Classifier, MLP có khả năng học các quan hệ phi tuyến phức tạp hơn.
Tuy nhiên, do dữ liệu đầu vào đã được Flatten, mô hình không có cơ chế chuyên biệt để khai thác tính cục bộ và cấu trúc không gian của ảnh như CNN.

3.3.3 CNN Classifier

CNN được xây dựng nhằm khai thác trực tiếp cấu trúc không gian của ảnh thông qua các lớp Convolution và Pooling.

Kiến trúc CNN trong thực nghiệm gồm hai khối trích xuất đặc trưng và một phần phân loại sử dụng các lớp Linear.

3.3.3.1 Trích xuất đặc trưng

Khối Convolution đầu tiên sử dụng 16 kernel kích thước 3×3, stride bằng 1 và padding bằng 1.
Với ảnh đầu vào có kích thước $1 \times 28 \times 28$, kết quả sau Convolution là:

$$1 \times 28 \times 28 \to 16 \times 28 \times 28 \tag{3.12}$$

Hàm ReLU được sử dụng để bổ sung tính phi tuyến, sau đó MaxPooling với kernel 2×2 và stride bằng 2 làm giảm kích thước không gian:

$$16 \times 28 \times 28 \to 16 \times 14 \times 14 \tag{3.13}$$

Khối Convolution thứ hai sử dụng 32 kernel kích thước 3×3, stride bằng 1 và padding bằng 1.

Sau Convolution, ReLU và MaxPooling lần thứ hai, kích thước tensor trở thành:

$$32 \times 7 \times 7 \tag{3.14}$$

Như vậy, quá trình trích xuất đặc trưng có thể tóm tắt:

$$\begin{aligned} 1 \times 28 \times 28 &\to 16 \times 28 \times 28 \\ &\to 16 \times 14 \times 14 \\ &\to 32 \times 14 \times 14 \\ &\to 32 \times 7 \times 7 \end{aligned} \tag{3.15}$$

3.3.3.2 Phân loại đặc trưng

Sau quá trình trích xuất, tensor $32 \times 7 \times 7$ được Flatten thành vector có 1.568 phần tử:

$$32 \times 7 \times 7 = 1\,568 \tag{3.16}$$

Vector này được đưa qua các lớp Linear có kích thước:

$$1568 \to 128 \to 64 \to 10$$

Hàm ReLU được áp dụng giữa các lớp ẩn. Lớp cuối cùng tạo ra 10 logits phục vụ phân loại. Tổng số tham số của CNN là:

$$P_{\text{CNN}} = 214\,538 \tag{3.17}$$

So với hai mô hình trước, CNN có khả năng khai thác đặc trưng cục bộ và chia sẻ trọng số tại nhiều vị trí khác nhau trên ảnh.

Điều này giúp CNN học các biểu diễn phù hợp hơn với cấu trúc của dữ liệu ảnh.

3.4 Kết quả thực nghiệm

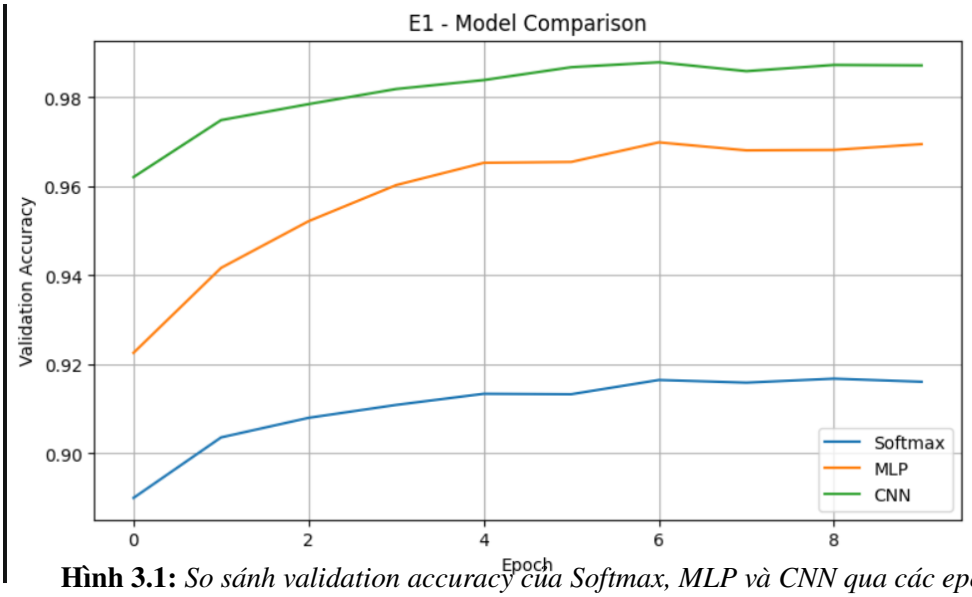
Ba mô hình được huấn luyện trên cùng tập dữ liệu và cấu hình đã trình bày trong Chương 2.

Bảng 3.1 tổng hợp độ chính xác trên tập test, số tham số và thời gian huấn luyện của từng mô hình.

Bảng 3.1: *Kết quả thực nghiệm E1*

Mô hình	Accuracy (%)	Parameters	Time (s)
Softmax	92,51	7 850	61,97
MLP	97,24	109 386	63,67
CNN	98,90	214 538	70,94

Hình 3.1 thể hiện sự thay đổi của validation accuracy trong 10 epoch huấn luyện đối với ba mô hình.



Hình 3.1: *So sánh validation accuracy của Softmax, MLP và CNN qua các epoch*

3.4.1 Nhận xét kết quả

Từ Hình 3.1, có thể thấy cả ba mô hình đều cải thiện độ chính xác trong những epoch đầu tiên, sau đó dần ổn định ở các mức khác nhau.

Softmax có tốc độ cải thiện chậm hơn và đạt mức validation accuracy thấp nhất. MLP đạt độ chính xác cao hơn đáng kể, cho thấy hiệu quả của việc bổ sung các phép biến đổi phi tuyến.

CNN đạt validation accuracy cao nhất ngay từ những epoch đầu và tiếp tục cải thiện trước khi hội tụ gần mức 99%. Điều này cho thấy kiến trúc CNN có khả năng học đặc trưng phù hợp với dữ liệu ảnh MNIST.

Kết quả trên tập test cũng thể hiện xu hướng tương tự. Softmax đạt 92,51%, MLP đạt 97,24%, trong khi CNN đạt 98,90%, cao hơn MLP 1,66 điểm phần trăm và Softmax 6,39 điểm phần trăm.

Về độ phức tạp, Softmax có ít tham số nhất với 7.850 tham số, trong khi MLP và CNN lần lượt có 109.386 và 214.538 tham số. Tuy nhiên, thời gian huấn luyện không tăng tương ứng với tỷ lệ số tham số.

Cụ thể, thời gian huấn luyện của Softmax và MLP lần lượt là 61,97 và 63,67 giây, còn CNN cần 70,94 giây. CNN có thời gian huấn luyện lớn hơn, nhưng mức chênh lệch tương đối nhỏ so với lợi ích về độ chính xác trong thí nghiệm này.

Nhìn chung, cả biểu đồ validation accuracy và kết quả trên tập test đều cho thấy CNN đạt hiệu quả phân loại tốt nhất trong ba kiến trúc được khảo sát. Tuy nhiên, sự khác biệt về số tham số và cấu trúc tính toán cũng cần được xem xét khi so sánh hiệu quả tổng thể.

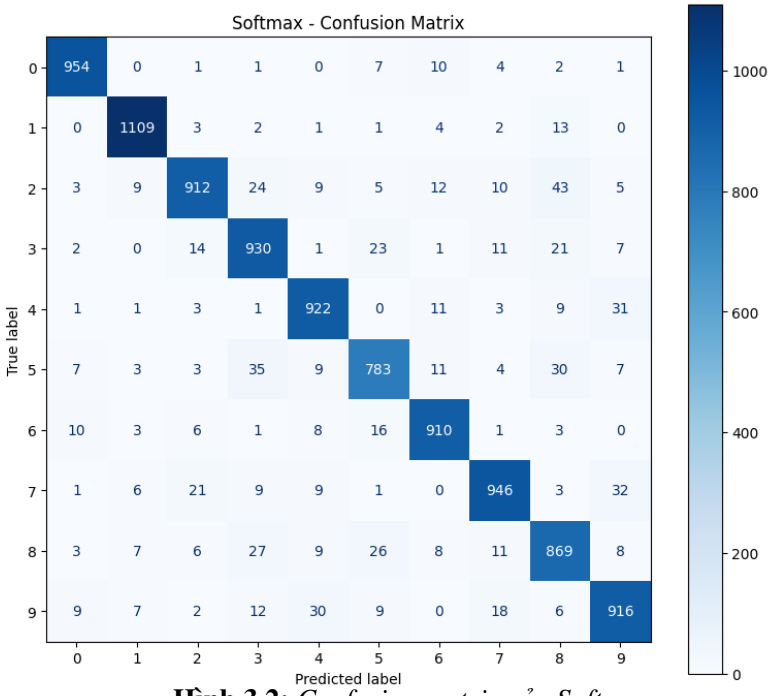
3.5 Phân tích lỗi và thảo luận

Để đánh giá chi tiết hơn khả năng phân loại, confusion matrix và một số trường hợp dự đoán sai được sử dụng để phân tích lỗi của từng mô hình.

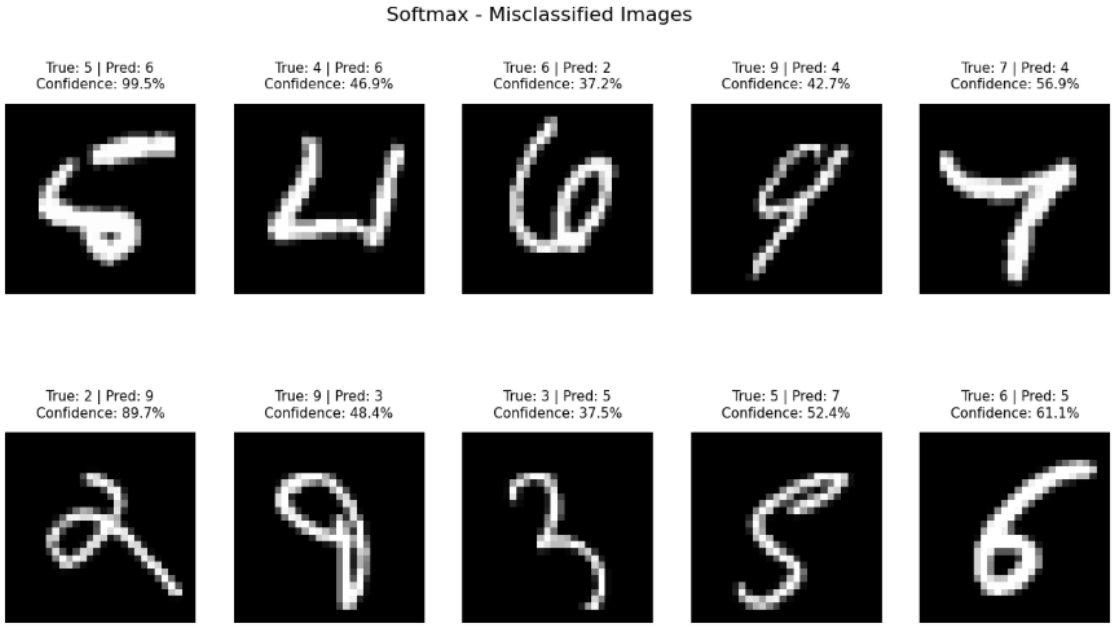
Trong confusion matrix, hàng biểu diễn nhãn thực tế và cột biểu diễn nhãn được dự đoán. Các giá trị trên đường chéo chính tương ứng với dự đoán đúng, trong khi các giá trị ngoài đường chéo thể hiện sự nhầm lẫn giữa các lớp.

3.5.1 Softmax Classifier

Hình 3.2 cho thấy Softmax vẫn nhận diện đúng phần lớn các chữ số, tuy nhiên xuất hiện nhiều giá trị ngoài đường chéo chính, đặc biệt ở một số lớp có hình dạng dễ nhầm lẫn.



Hình 3.2: *Confusion matrix của Softmax*




```
Total misclassified images: 749
```

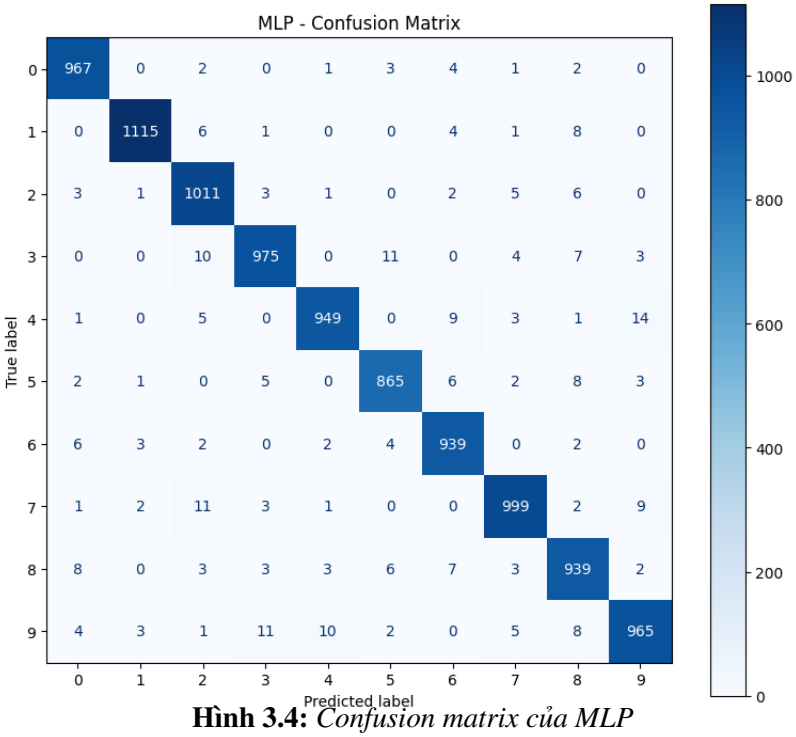
Hình 3.3: *Một số trường hợp dự đoán sai của Softmax*

Chẳng hạn, chữ số 2 bị dự đoán thành 8 trong 43 trường hợp, chữ số 5 bị dự đoán thành 3 trong 35 trường hợp, và chữ số 4 bị dự đoán thành 9 trong 31 trường hợp.

Các ví dụ trong Hình 3.3 minh họa những trường hợp mô hình không phân biệt chính xác các đặc điểm hình dạng của chữ số.

Tổng cộng, Softmax phân loại sai 749 trong 10.000 ảnh kiểm thử. Kết quả này phản ánh hạn chế của mô hình tuyến tính khi xử lý những biến thể phức tạp của chữ viết tay.

3.5.2 MLP Classifier



Hình 3.4: *Confusion matrix của MLP*

So với Softmax, confusion matrix của MLP trong Hình 3.4 có các giá trị tập trung rõ hơn trên đường chéo chính, cho thấy độ chính xác được cải thiện.

Các trường hợp nhầm lẫn phổ biến cũng giảm đáng kể. Ví dụ, số trường hợp chữ số 2 bị dự đoán thành 8 giảm từ 43 xuống còn 6.

Tuy nhiên, mô hình vẫn gặp khó khăn ở một số cặp chữ số như 4 và 9, 3 và 5, hoặc 7 và 2.

Các mẫu trong Hình 3.5 cho thấy việc sử dụng nhiều lớp ẩn không loại bỏ hoàn toàn các sai sót do hình dạng chữ số không rõ ràng hoặc có nét viết tương đồng.

MLP phân loại sai 276 ảnh, ít hơn 473 ảnh so với Softmax. Kết quả này cho thấy các phép biến đổi phi tuyến giúp cải thiện đáng kể khả năng phân biệt những mẫu chữ số phức tạp.

3.5.3 CNN Classifier

Hình 3.6 cho thấy các giá trị dự đoán của CNN tập trung mạnh nhất trên đường chéo chính, với rất ít trường hợp nhầm lẫn giữa các lớp.

Chẳng hạn, chữ số 2 chỉ bị dự đoán thành 8 trong một trường hợp, thấp hơn đáng kể so với Softmax và MLP.

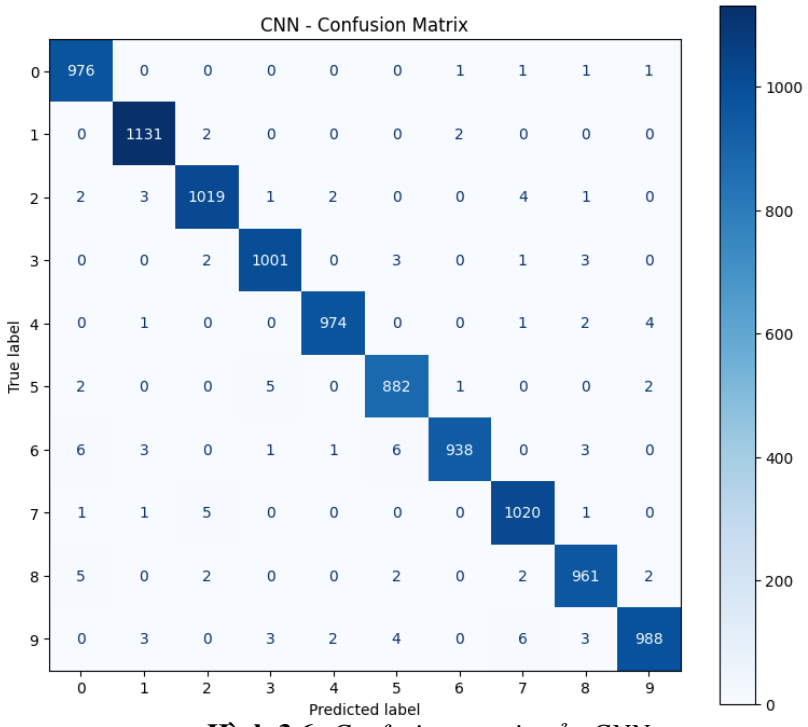
Dù vậy, CNN vẫn xuất hiện một số lỗi như chữ số 6 bị nhận nhầm thành 0 hoặc chữ số 5 thành 3.

Quan sát Hình 3.7, một số ảnh dự đoán sai có nét viết biến dạng, chồng chéo hoặc không thể hiện rõ hình dạng đặc trưng của chữ số tương ứng.

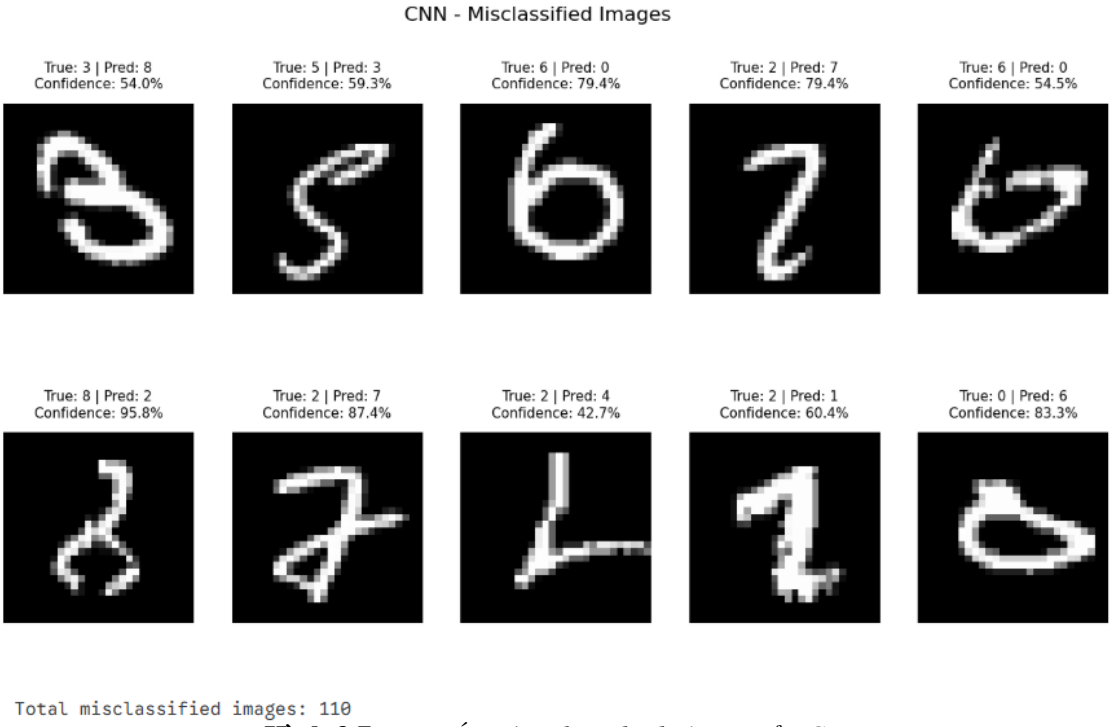



```
Total misclassified images: 276
```

Hình 3.5: *Một số trường hợp dự đoán sai của MLP*



Hình 3.6: *Confusion matrix của CNN*




```
Total misclassified images: 110
```

Hình 3.7: *Một số trường hợp dự đoán sai của CNN*

Đáng chú ý, mô hình có những dự đoán sai với độ tin cậy tương đối cao, cho thấy xác suất dự đoán lớn không đồng nghĩa với kết quả luôn chính xác.

CNN chỉ phân loại sai 110 ảnh trong tập test, ít hơn 166 ảnh so với MLP và 639 ảnh so với Softmax.

3.6 Kết luận E1

Thực nghiệm E1 cho thấy kiến trúc mạng có ảnh hưởng rõ rệt đến hiệu quả phân loại ảnh MNIST.

Softmax Classifier có ưu điểm đơn giản, ít tham số và phù hợp làm baseline.

MLP cải thiện khả năng phân loại nhờ các lớp ẩn và hàm kích hoạt phi tuyến.

CNN đạt kết quả tốt nhất trong ba mô hình nhờ khai thác đặc trưng không gian thông qua Convolution.

Kết quả này tạo cơ sở để tiếp tục nghiên cứu các phương pháp biểu diễn và xử lý ảnh khác trong E2 và E3.

Chương 4

E2: Phân loại ảnh bằng Transformer và Self-Attention

4.1 Mục tiêu

Bài tập E2 tập trung nghiên cứu khả năng ứng dụng Transformer vào bài toán phân loại ảnh MNIST thông qua cơ chế Multi-Head Self-Attention (MSA).

Khác với CNN, Transformer không trực tiếp xử lý ảnh bằng các phép tích chập xuyên suốt kiến trúc, mà tiếp nhận dữ liệu dưới dạng một chuỗi token. Mối quan hệ giữa các token được học thông qua cơ chế Self-Attention.

Thực nghiệm gồm hai nội dung chính: so sánh các phương pháp biểu diễn ảnh thành token và so sánh MSA được hiện thực thủ công với MSA có sẵn trong PyTorch.

4.2 Biểu diễn ảnh thành chuỗi token

Ảnh MNIST ban đầu có kích thước $1 \times 28 \times 28$. Để đưa vào Transformer, ảnh cần được chuyển thành chuỗi gồm N token, mỗi token có số chiều biểu diễn $D = 64$.

Đầu ra của quá trình embedding có dạng:

$$X_{\text{embed}} \in \mathbb{R}^{N \times 64} \tag{4.1}$$

Ba phương pháp tokenization được khảo sát là Patch Embedding, Row Embedding và CNN Embedding.

4.2.1 Patch Embedding

Patch Embedding chia ảnh thành các vùng hình vuông không chồng lấp, gọi là patch.

Trong thực nghiệm, mỗi patch có kích thước 7×7. Với ảnh 28×28, số patch thu được là:

$$N = \frac{28}{7} \times \frac{28}{7} = 16 \tag{4.2}$$

Mỗi patch chứa 49 giá trị pixel và được ánh xạ thành vector 64 chiều thông qua một phép chiếu tuyến tính có tham số học được.

Trong mô hình, phép chiếu này được thực hiện bằng Convolution với kernel 7×7 và stride bằng 7, tương đương với việc áp dụng cùng một phép chiếu tuyến tính cho từng patch.

Quá trình biến đổi kích thước được biểu diễn:

$$1 \times 28 \times 28 \rightarrow 64 \times 4 \times 4 \rightarrow 16 \times 64 \tag{4.3}$$

Như vậy, ảnh được biểu diễn thành 16 token, mỗi token chứa thông tin của một vùng 7×7 pixel.

Phương pháp này giữ được thông tin theo từng vùng ảnh nhưng không trực tiếp mô hình hóa quan hệ giữa các pixel thuộc những patch khác nhau. Những mối quan hệ đó sẽ được học ở bước Self-Attention.

4.2.2 Row Embedding

Row Embedding xem mỗi hàng của ảnh là một token.

Với ảnh MNIST, mỗi hàng chứa 28 giá trị pixel. Do ảnh có 28 hàng, chuỗi đầu vào gồm 28 token.

Mỗi hàng được ánh xạ từ vector 28 chiều thành vector 64 chiều thông qua một lớp Linear.

Quá trình biến đổi có dạng:

$$1 \times 28 \times 28 \rightarrow 28 \times 28 \rightarrow 28 \times 64 \tag{4.4}$$

Khác với Patch Embedding, Row Embedding không chia ảnh thành các vùng hình vuông mà giữ nguyên thứ tự các hàng.

Cách biểu diễn này cho phép Transformer học mối quan hệ giữa những phần khác nhau theo chiều dọc của ảnh.

Tuy nhiên, thông tin của mỗi hàng đã được gộp thành một vector token nên mô hình không trực tiếp thực hiện Attention giữa từng pixel riêng lẻ.

4.2.3 CNN Embedding

CNN Embedding sử dụng các lớp Convolution để trích xuất đặc trưng trước khi chuyển chúng thành chuỗi token.

Trong thực nghiệm, khối embedding gồm hai lớp Convolution có kernel 3×3. Lớp đầu tiên tạo ra 256 feature map, lớp thứ hai tạo ra 64 feature map. Mỗi lớp Convolution được theo sau bởi ReLU và MaxPooling kích thước 2×2.

Quá trình biến đổi được mô tả:

$$\begin{aligned} 1 \times 28 \times 28 &\rightarrow 256 \times 28 \times 28 \\ &\rightarrow 256 \times 14 \times 14 \\ &\rightarrow 64 \times 14 \times 14 \\ &\rightarrow 64 \times 7 \times 7 \end{aligned} \tag{4.5}$$

Sau đó, feature map được chuyển thành chuỗi token:

$$64 \times 7 \times 7 \rightarrow 49 \times 64 \tag{4.6}$$

Mỗi token tương ứng với một vị trí trên feature map có 64 kênh đặc trưng.

Khác với Patch và Row Embedding, các token ở đây không chỉ chứa giá trị pixel ban đầu mà còn chứa đặc trưng đã được học thông qua các phép tích chập.

Nhờ đó, Transformer có thể khai thác đồng thời đặc trưng cục bộ từ CNN và mối quan hệ giữa các token thông qua Self-Attention.

4.2.4 So sánh các phương pháp tokenization

Bảng 4.1: *So sánh ba phương pháp tokenization*

Phương pháp	Số token	Embedding dim	Đầu ra
Patch	16	64	16×64
Row	28	64	28×64
CNN	49	64	49×64

Ba phương pháp tạo ra những chuỗi token có số lượng và đặc điểm khác nhau, trong khi cùng sử dụng embedding dimension bằng 64.

Sự khác biệt này ảnh hưởng đến lượng thông tin được biểu diễn trong mỗi token, số phép tính Attention và khả năng học đặc trưng của mô hình.

4.3 Cơ chế Multi-Head Self-Attention

4.3.1 Nguyên lý Self-Attention

Self-Attention cho phép mỗi token tổng hợp thông tin từ những token khác trong cùng một chuỗi dựa trên mức độ liên quan được mô hình học.

Từ embedding đầu vào X, ba phép chiếu tuyến tính tạo ra các ma trận Query, Key và Value:

$$Q = XW_Q \tag{4.7}$$
$$K = XW_K \tag{4.8}$$
$$V = XW_V \tag{4.9}$$

Mức độ liên quan giữa các token được xác định thông qua tích vô hướng giữa Query và Key.

Đầu ra Attention được tính theo công thức:

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{4.10}$$

Trong đó d_k là số chiều của Key trong mỗi attention head.

Phép chia cho $\sqrt{d_k}$ giúp hạn chế giá trị tích vô hướng trở nên quá lớn khi số chiều biểu diễn tăng.

Softmax chuyển các attention scores thành trọng số, sau đó những trọng số này được sử dụng để tổng hợp thông tin từ Value.

Nhờ đó, mỗi token đầu ra có thể chứa thông tin từ nhiều vị trí khác nhau trong ảnh.

4.3.2 Multi-Head Self-Attention

Thay vì chỉ sử dụng một cơ chế Attention, MSA chia không gian embedding thành nhiều attention head hoạt động song song.

Trong thực nghiệm, số chiều embedding là $D = 64$ và số head là $H = 4$. Do đó, số chiều của mỗi head là:

$$d_k = \frac{D}{H} = \frac{64}{4} = 16 \tag{4.11}$$

Mỗi head thực hiện Self-Attention trên những biểu diễn được học riêng.

Các kết quả được ghép lại và đưa qua một phép chiếu tuyến tính:

$$\text{MSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_4) W_O \tag{4.12}$$

Việc sử dụng nhiều head cho phép mô hình học nhiều dạng quan hệ khác nhau giữa các token, thay vì chỉ dựa vào một tập trọng số Attention duy nhất.

4.3.3 MSA tự xây dựng và MSA của PyTorch

Bài tập sử dụng hai cách hiện thực MSA.

Ở phiên bản tự xây dựng, các bước chiếu Query, Key, Value, tách attention head, tính attention scores, áp dụng Softmax và tổng hợp Value được triển khai trực tiếp từ các phép toán tensor.

Phiên bản thứ hai sử dụng module MultiheadAttention của PyTorch với cùng embedding dimension và số attention head.

Hai phiên bản có cùng nguyên lý toán học và số lượng tham số trong cấu hình thực nghiệm. Tuy nhiên, cách tổ chức và tối ưu các phép toán có thể khác nhau, dẫn đến sự khác biệt về thời gian thực thi.

4.4 Kiến trúc Transformer Classifier

Các mô hình trong E2 sử dụng cùng kiến trúc Transformer Classifier, chỉ thay đổi phương pháp embedding hoặc phiên bản MSA.

Đầu tiên, ảnh được chuyển thành chuỗi token bằng một trong ba phương pháp đã trình bày.

Sau đó, mô hình cộng positional embedding có thể học được vào mỗi token để bổ sung thông tin vị trí.

$$X_0 = X_{\text{embed}} + E_{\text{pos}} \tag{4.13}$$

Chuỗi token được đưa qua một Transformer Block gồm MSA, LayerNorm, kết nối residual và mạng FFN.

Trong thực nghiệm, mô hình sử dụng một Transformer Block với FFN có hai lớp Linear:

$$64 \to 128 \to 64 \tag{4.14}$$

Hàm kích hoạt ReLU được sử dụng giữa hai lớp Linear.

Các kết nối residual giúp bảo toàn thông tin đầu vào và hỗ trợ quá trình lan truyền gradient.

Sau Transformer Block, Mean Pooling được sử dụng để tổng hợp các token thành một vector biểu diễn duy nhất:

$$h = \frac{1}{N} \sum_{i=1}^{N} X_i \tag{4.15}$$

Vector này đi qua LayerNorm và lớp Linear cuối cùng để tạo 10 logits tương ứng với các lớp chữ số MNIST.

Như vậy, luồng xử lý tổng quát của mô hình là:

Image → Embedding → Positional Embedding → Transformer Block → Mean Pooling → Classification

4.5 Kết quả thực nghiệm

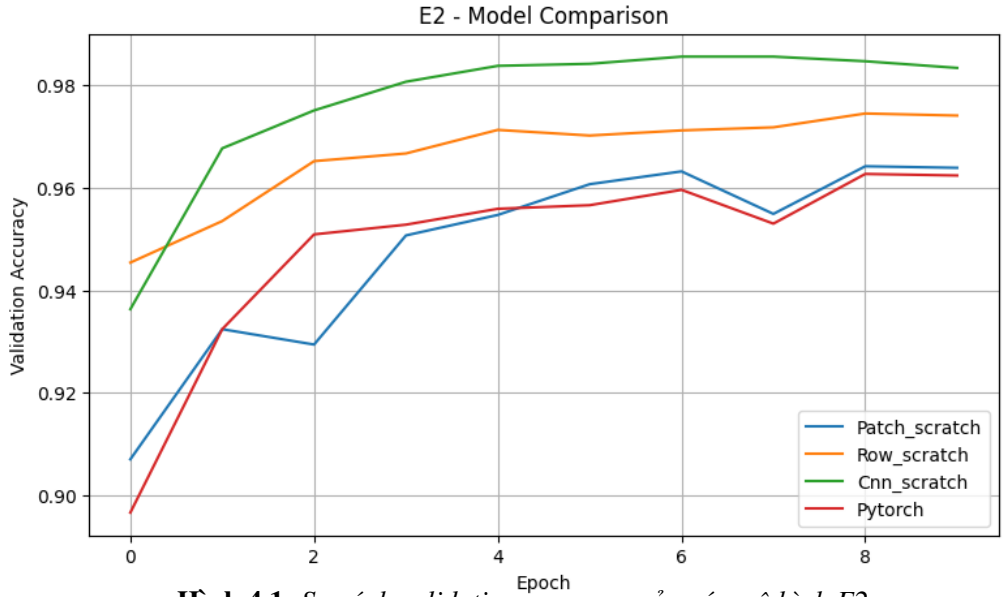
Bốn cấu hình được huấn luyện với cùng tập dữ liệu và cấu hình cơ bản đã trình bày trong Chương 2.

Ba cấu hình đầu sử dụng MSA tự xây dựng với các phương pháp embedding khác nhau. Cấu hình thứ tư sử dụng Patch Embedding kết hợp MSA của PyTorch.

Bảng 4.2: *Kết quả thực nghiệm E2*

Mô hình	Accuracy (%)	Parameters	Time (s)
Patch + Scratch	96,80	38 474	77,39
Row + Scratch	97,76	37 898	75,18
CNN + Scratch	98,65	187 466	117,02
Patch + PyTorch	96,79	38 474	74,47

Hình 4.1 thể hiện validation accuracy của bốn cấu hình trong 10 epoch huấn luyện.



Hình 4.1: *So sánh validation accuracy của các mô hình E2*

4.5.1 Ảnh hưởng của phương pháp tokenization

Kết quả cho thấy CNN Embedding đạt accuracy cao nhất với 98,65%, tiếp theo là Row Embedding với 97,76% và Patch Embedding với 96,80%.

Khi sử dụng cùng MSA tự xây dựng, Row Embedding đạt accuracy cao hơn Patch Embedding 0,96 điểm phần trăm, trong khi số lượng tham số thấp hơn.

Điều này cho thấy cách chia ảnh thành token có thể ảnh hưởng đáng kể đến hiệu quả phân loại, ngay cả khi kiến trúc Transformer phía sau được giữ nguyên.

CNN Embedding đạt accuracy cao hơn Patch Embedding 1,85 điểm phần trăm. Tuy nhiên, mô hình này có 187.466 tham số, lớn hơn đáng kể so với 38.474 tham số của Patch Embedding.

Do đó, kết quả tốt hơn không thể chỉ được giải thích bởi phương pháp tokenization mà còn chịu ảnh hưởng từ khả năng trích xuất đặc trưng và quy mô lớn hơn của CNN Embedding.

Về thời gian, CNN Embedding cần 117,02 giây, cao hơn hai phương pháp còn lại. Nguyên nhân có thể đến từ khối Convolution bổ sung và số token lớn hơn được đưa vào Self-Attention.

4.5.2 So sánh hai phiên bản MSA

Hai mô hình Patch + Scratch và Patch + PyTorch có cùng phương pháp tokenization, embedding dimension, số attention head và số tham số.

Kết quả accuracy lần lượt là 96,80% và 96,79%, chênh lệch chỉ 0,01 điểm phần trăm.

Điều này cho thấy hai cách hiện thực MSA đạt hiệu quả phân loại gần như tương đương trong thí nghiệm.

Tuy nhiên, thời gian huấn luyện của phiên bản PyTorch là 74,47 giây, thấp hơn 77,39 giây của phiên bản tự xây dựng.

Sự khác biệt này phù hợp với khả năng tối ưu thực thi của các module có sẵn trong framework.

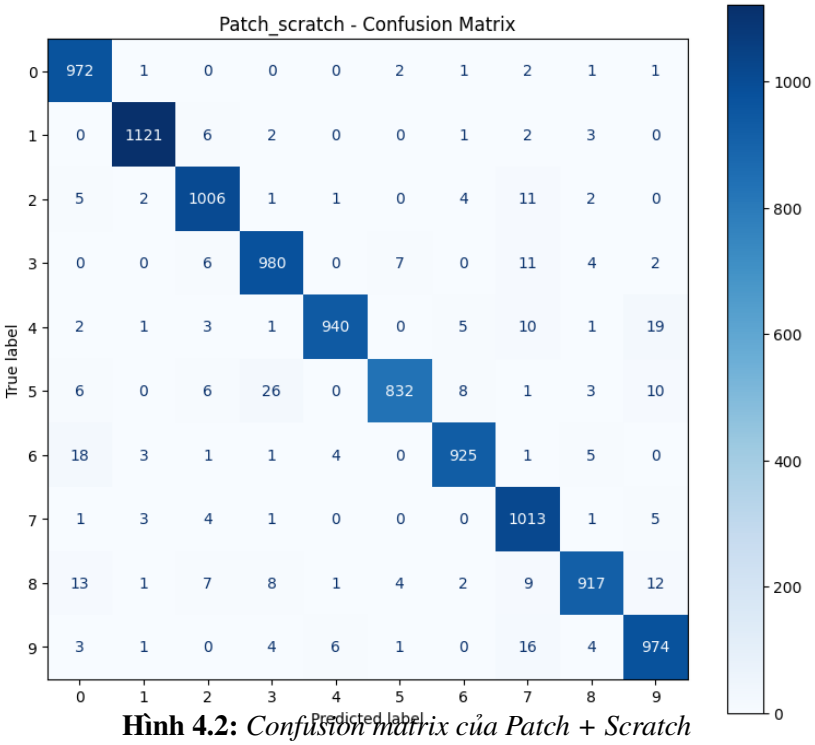
Dù vậy, kết quả chỉ được đo trong một lần chạy, do đó chưa đủ cơ sở để khẳng định sự khác biệt về tốc độ hay accuracy là ổn định trong mọi điều kiện.

4.6 Phân tích lỗi và thảo luận

Để phân tích chi tiết khả năng phân loại, confusion matrix và các ảnh dự đoán sai được sử dụng cho cả bốn mô hình.

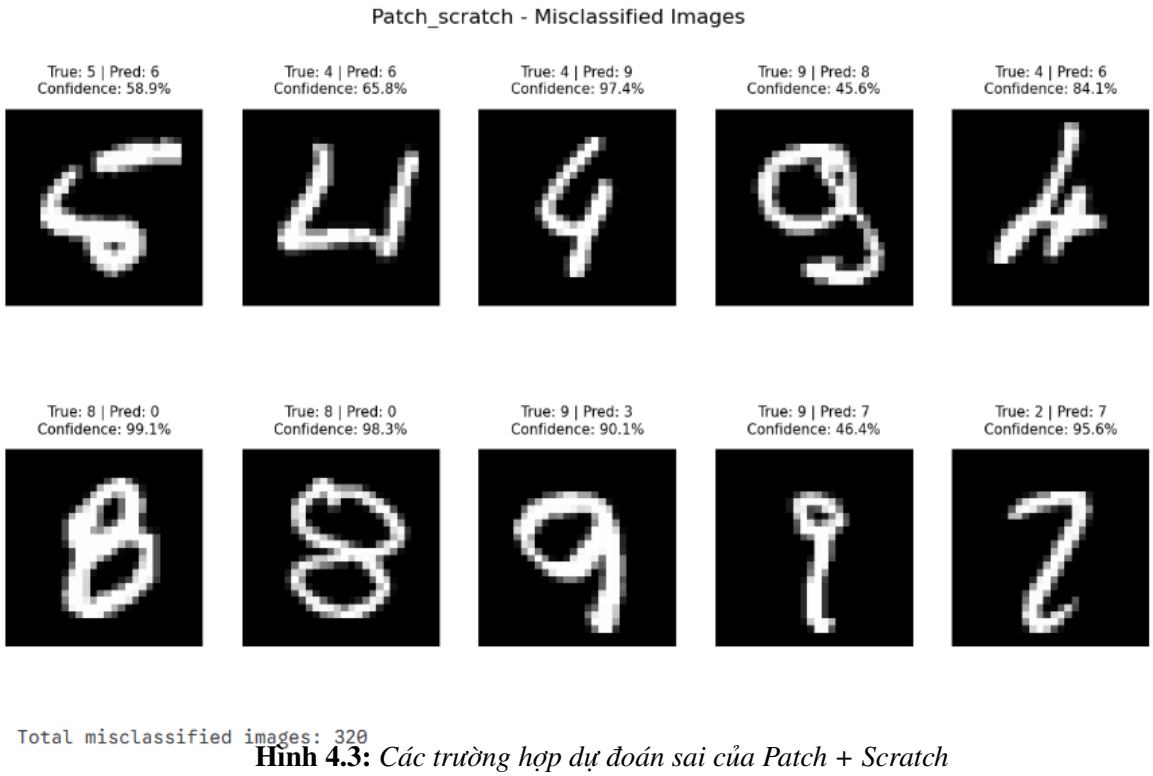
Các hình cho phép quan sát mức độ nhầm lẫn giữa những chữ số, đồng thời bổ sung thông tin cho kết quả accuracy tổng thể.

4.6.1 Patch Embedding với Scratch MSA



Hình 4.2: *Confusion matrix của Patch + Scratch*

Hình 4.2 cho thấy phần lớn dự đoán tập trung trên đường chéo chính của confusion matrix, nhưng vẫn tồn tại những trường hợp nhầm lẫn giữa các chữ số.



Hình 4.3: *Các trường hợp dự đoán sai của Patch + Scratch*

Patch Embedding chia ảnh thành 16 vùng riêng biệt. Cách biểu diễn này cho phép mô hình học mối quan hệ giữa các vùng ảnh thông qua Attention, nhưng mỗi token ban đầu chỉ chứa thông tin từ một patch.

Các ví dụ trong Hình 4.3 minh họa những trường hợp dự đoán sai, có thể liên quan đến hình dạng chữ số khó phân biệt hoặc thông tin đặc trưng chưa được mô hình khai thác đầy đủ.

Tổng cộng, mô hình phân loại sai 320 trong 10.000 ảnh kiểm thử.

4.6.2 Row Embedding với Scratch MSA

So với Patch Embedding, Row Embedding cho kết quả chính xác hơn và có ít giá trị nhầm lẫn ngoài đường chéo chính hơn khi xét tổng thể.

Việc biểu diễn mỗi hàng ảnh thành một token cho phép mô hình tiếp nhận thông tin theo những lát cắt ngang của chữ số.

Cách biểu diễn này có thể phù hợp với các đặc điểm nét viết trong MNIST, giúp mô hình học mối liên hệ giữa các hàng ảnh hiệu quả hơn trong cấu hình thực nghiệm.

Các mẫu trong Hình 4.5 cho thấy mô hình vẫn có thể dự đoán sai đối với những chữ số khó phân biệt.

Tổng số ảnh phân loại sai là 224, ít hơn 96 ảnh so với Patch Embedding.

4.6.3 CNN Embedding với Scratch MSA

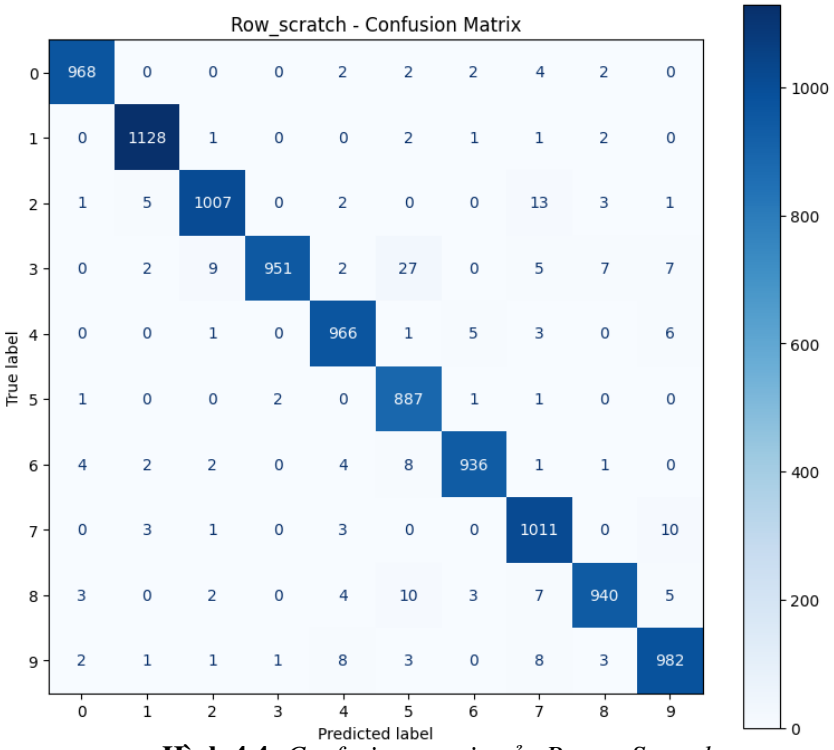
Hình 4.6 cho thấy các giá trị tập trung mạnh nhất trên đường chéo chính trong bốn cấu hình được khảo sát.

Khác với Patch và Row Embedding, CNN Embedding thực hiện trích xuất đặc trưng cục bộ trước khi đưa các token vào Transformer.

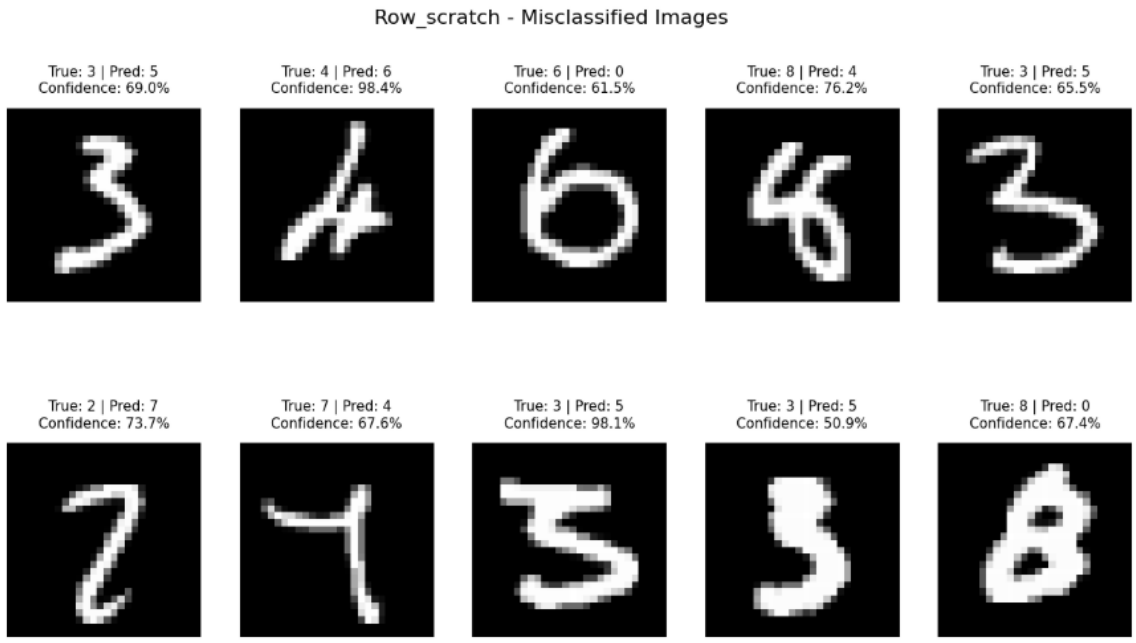
Điều này có thể giúp mô hình biểu diễn tốt hơn những nét viết và cấu trúc hình dạng của chữ số.

Tuy nhiên, các mẫu trong Hình 4.7 cho thấy một số trường hợp dự đoán sai vẫn tồn tại.

CNN Embedding phân loại sai 135 ảnh, thấp nhất trong bốn cấu hình E2.

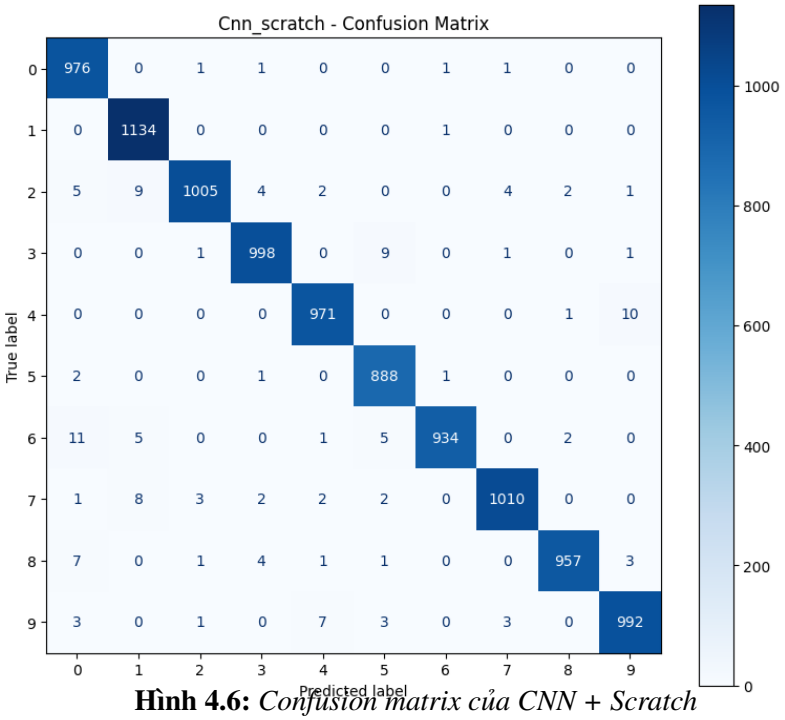


Hình 4.4: *Confusion matrix của Row + Scratch*

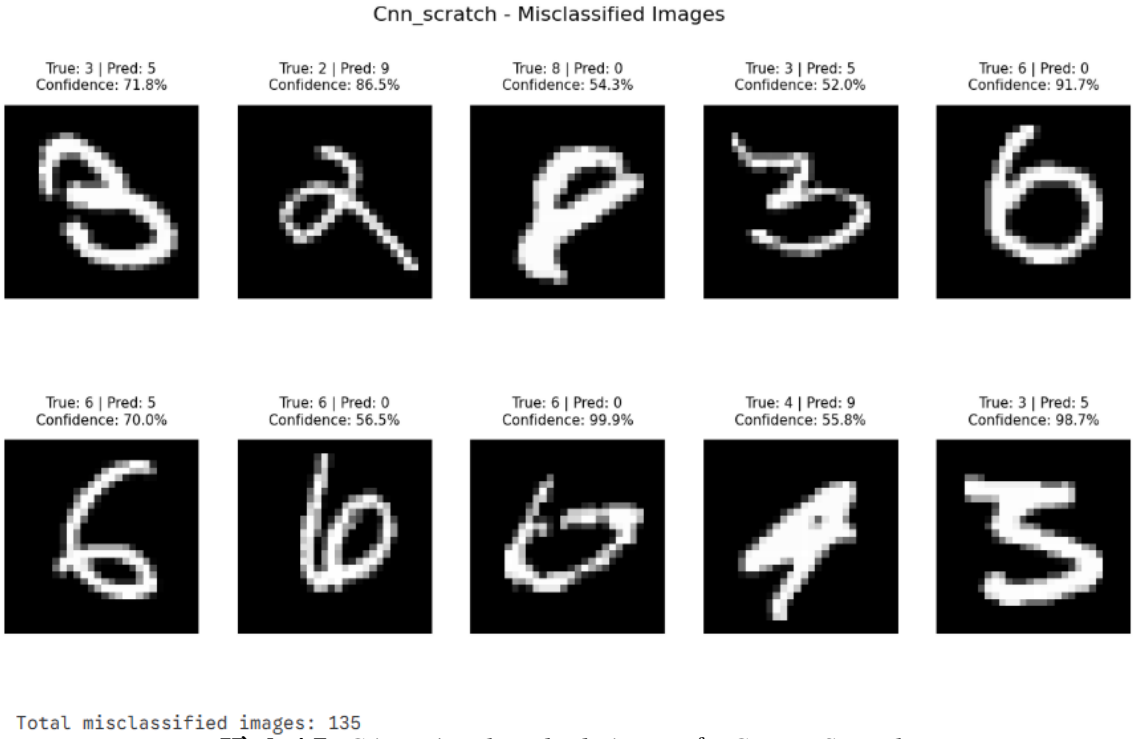



```
Total misclassified images: 224
```

Hình 4.5: *Các trường hợp dự đoán sai của Row + Scratch*



Hình 4.6: *Confusion matrix của CNN + Scratch*

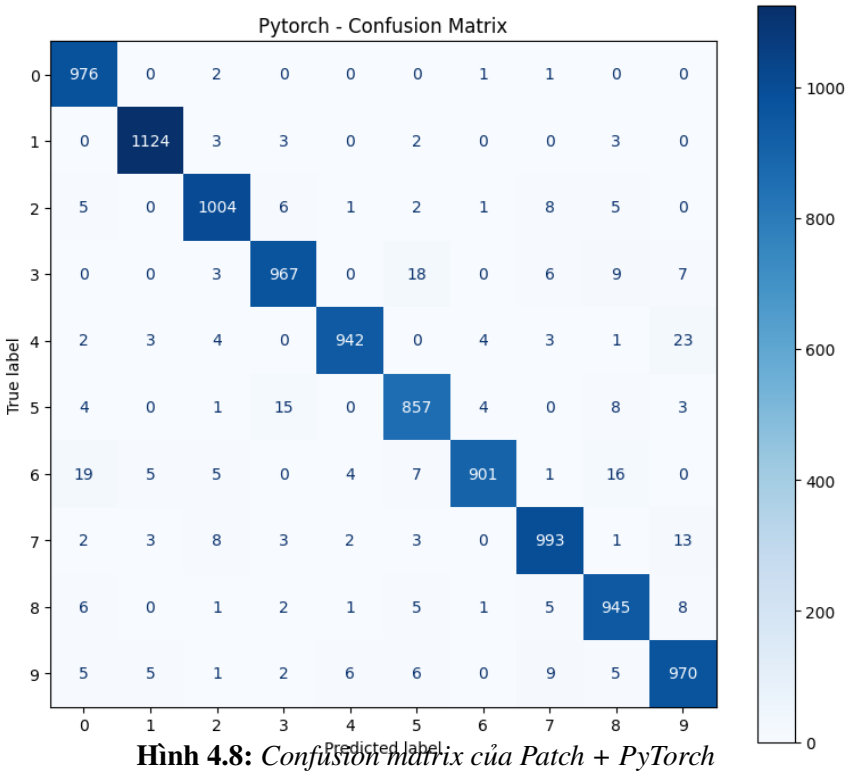



```
Total misclassified images: 135
```

Hình 4.7: *Các trường hợp dự đoán sai của CNN + Scratch*

Kết quả này cho thấy lợi ích của việc kết hợp CNN và Self-Attention, nhưng cũng cần xem xét chi phí tính toán và số tham số lớn hơn.

4.6.4 Patch Embedding với PyTorch MSA



Hình 4.8: *Confusion matrix của Patch + PyTorch*

Confusion matrix của phiên bản PyTorch MSA có phân bố dự đoán gần tương đương với phiên bản MSA tự xây dựng khi sử dụng cùng Patch Embedding.

Các trường hợp dự đoán sai trong Hình 4.9 tiếp tục thể hiện những khó khăn trong việc phân biệt các chữ số có hình dạng tương tự.

Tổng cộng, mô hình phân loại sai 321 ảnh, chỉ nhiều hơn phiên bản Scratch MSA một ảnh.

Kết quả cho thấy việc thay thế cách hiện thực MSA không làm thay đổi đáng kể hiệu quả phân loại trong cấu hình thí nghiệm này.

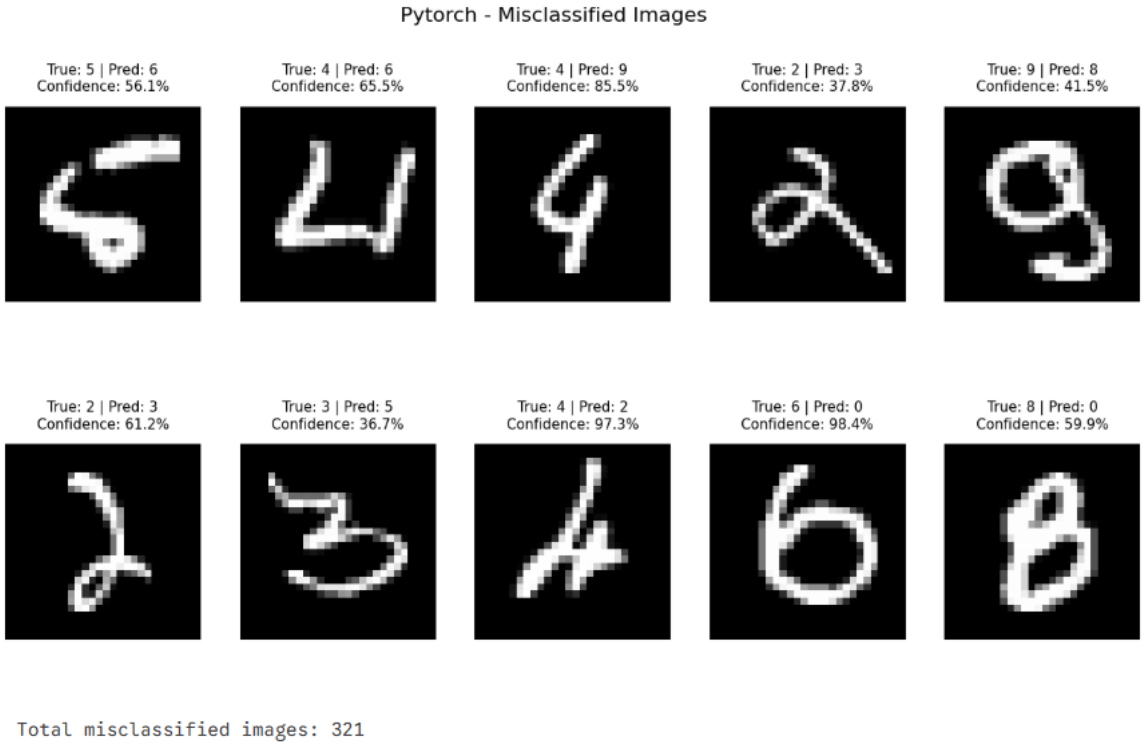
4.7 Kết luận E2

Thực nghiệm E2 cho thấy Transformer có thể được áp dụng hiệu quả vào bài toán phân loại ảnh MNIST bằng cách chuyển dữ liệu ảnh thành chuỗi token.

Phương pháp tokenization ảnh hưởng rõ rệt đến kết quả phân loại. Trong ba phương pháp được khảo sát, CNN Embedding đạt accuracy cao nhất, trong khi Row Embedding đạt kết quả tốt với số tham số thấp hơn.

Hai phiên bản MSA tự xây dựng và MSA của PyTorch đạt kết quả gần tương đương khi sử dụng cùng cấu hình Patch Embedding.

Những kết quả này cho thấy chất lượng của mô hình Transformer không chỉ phụ thuộc vào cơ chế Self-Attention mà còn chịu ảnh hưởng đáng kể từ cách biểu diễn dữ liệu trước khi đưa vào mô hình.




```
Total misclassified images: 321
```

Hình 4.9: *Các trường hợp dự đoán sai của Patch + PyTorch*

Chương 5

E3: Phân loại ảnh bằng LSTM và GRU

5.1 Mục tiêu

Bài tập E3 nghiên cứu khả năng ứng dụng các mạng nơ-ron hồi quy vào bài toán phân loại ảnh MNIST thông qua việc biểu diễn ảnh dưới dạng chuỗi.

Khác với CNN khai thác đặc trưng không gian bằng các phép tích chập, LSTM và GRU xử lý dữ liệu theo từng bước thời gian và tích lũy thông tin thông qua trạng thái ẩn.

Thực nghiệm tập trung so sánh khả năng phân loại của LSTM và GRU khi sử dụng cùng cách biểu diễn ảnh thành chuỗi, đồng thời đối chiếu với các mô hình MLP và CNN trong E1.

5.2 Biểu diễn ảnh thành chuỗi

5.2.1 Row Sequence

Trong thực nghiệm, cả LSTM và GRU đều sử dụng phương pháp Row Sequence, trong đó mỗi hàng của ảnh được xem như một bước thời gian.

Ảnh MNIST có kích thước 28×28 pixel và một kênh màu. Sau khi loại bỏ chiều kênh, ảnh được biểu diễn thành chuỗi:

$$X = (x_1, x_2, \ldots, x_{28}), \qquad x_t \in \mathbb{R}^{28} \tag{5.1}$$

Trong đó, x_t là vector chứa 28 giá trị pixel thuộc hàng thứ t của ảnh.
Như vậy, chuỗi đầu vào có:

- Số bước thời gian: $T = 28$.
- Số đặc trưng tại mỗi bước: 28.
- Kích thước chuỗi: 28×28.

Quá trình biểu diễn được mô tả:

$$1 \times 28 \times 28 \longrightarrow 28 \times 28 \tag{5.2}$$

Khác với tokenization trong E2, Row Sequence không sử dụng một lớp embedding để chiếu các hàng ảnh sang không gian đặc trưng mới. Mỗi hàng được đưa trực tiếp vào mạng hồi quy dưới dạng vector 28 chiều.

5.2.2 Cơ chế xử lý tuần tự

LSTM và GRU lần lượt tiếp nhận các hàng ảnh từ trên xuống dưới. Tại mỗi bước thời gian, mô hình kết hợp thông tin của hàng hiện tại với trạng thái đã được tích lũy từ các hàng trước đó.

Quá trình này có thể biểu diễn tổng quát:

$$h_t = f(x_t, h_{t-1}) \tag{5.3}$$

Trong đó h_t là trạng thái ẩn tại bước thời gian t và f biểu diễn phép cập nhật trạng thái của mô hình.

Sau khi xử lý toàn bộ 28 hàng, trạng thái ẩn cuối cùng h_{28} được sử dụng để phân loại chữ số.

Cách biểu diễn này cho phép mô hình học các mối quan hệ tuần tự giữa những phần khác nhau của ảnh.

Tuy nhiên, quan hệ không gian hai chiều được chuyển thành một thứ tự duyệt cố định, vì vậy mô hình không trực tiếp khai thác cấu trúc cục bộ theo cả hai chiều như CNN.

5.3 Kiến trúc LSTM

5.3.1 Nguyên lý hoạt động

LSTM là một dạng mạng hồi quy được thiết kế để cải thiện khả năng duy trì thông tin qua nhiều bước thời gian.

Khác với mạng RNN cơ bản, LSTM sử dụng trạng thái ô nhớ c_t và trạng thái ẩn h_t, kết hợp với các cổng điều khiển để lựa chọn thông tin cần ghi nhớ, cập nhật hoặc loại bỏ.

Ba cổng chính của LSTM gồm Forget Gate, Input Gate và Output Gate.

Forget Gate xác định phần thông tin từ trạng thái ô nhớ trước đó cần được giữ lại:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \tag{5.4}$$

Input Gate xác định lượng thông tin mới được đưa vào ô nhớ:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{5.5}$$

Thông tin ứng viên được tạo bởi:

$$\widetilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{5.6}$$

Trạng thái ô nhớ được cập nhật:

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t \tag{5.7}$$

Cuối cùng, Output Gate xác định thông tin được đưa ra trạng thái ẩn:

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{5.8}$$
$$h_t = o_t \odot \tanh(c_t) \tag{5.9}$$

Trong các công thức trên, σ là hàm Sigmoid và $\odot$ là phép nhân theo từng phần tử.

Nhờ cơ chế ô nhớ và các cổng điều khiển, LSTM có khả năng duy trì những thông tin quan trọng từ các bước trước để phục vụ quá trình xử lý những bước tiếp theo.

5.3.2 Kiến trúc LSTM Classifier

Trong thực nghiệm, mô hình LSTM gồm một lớp hồi quy với các thông số:

- Input size: 28.
- Hidden size: 128.
- Số lớp LSTM: 1.
- Sequence length: 28.
- Số lớp phân loại: 10.

Tại mỗi bước thời gian, LSTM tiếp nhận vector 28 chiều và tạo trạng thái ẩn có 128 chiều. Sau 28 bước, mô hình sử dụng trạng thái ẩn cuối cùng để biểu diễn toàn bộ ảnh:

$$h_{28} \in \mathbb{R}^{128} \tag{5.10}$$

Vector này được đưa qua lớp Linear để tạo ra 10 logits:

$$z = W_{\text{fc}} h_{28} + b_{\text{fc}}, \qquad z \in \mathbb{R}^{10} \tag{5.11}$$

Luồng xử lý tổng quát:

$$\text{Image} \rightarrow \text{Row Sequence} \rightarrow \text{LSTM} \rightarrow \text{Final Hidden State} \rightarrow \text{Linear} \rightarrow \text{Logits}$$

Mô hình có tổng cộng 82.186 tham số, bao gồm các trọng số của LSTM và lớp phân loại cuối cùng.

5.4 Kiến trúc GRU

5.4.1 Nguyên lý hoạt động

GRU cũng sử dụng cơ chế cổng điều khiển để xử lý thông tin tuần tự, nhưng có cấu trúc đơn giản hơn LSTM.

GRU không sử dụng trạng thái ô nhớ riêng biệt mà duy trì thông tin thông qua trạng thái ẩn h_t.

Mô hình gồm hai cổng chính: Reset Gate và Update Gate.

Reset Gate điều khiển mức độ sử dụng thông tin từ trạng thái trước đó khi tạo trạng thái ứng viên.

Update Gate điều khiển sự kết hợp giữa thông tin đã ghi nhớ và thông tin mới.

Hai cổng được xác định như sau:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \tag{5.12}$$
$$u_t = \sigma(W_u x_t + U_u h_{t-1} + b_u) \tag{5.13}$$

Trong đó r_t là Reset Gate và u_t là Update Gate.
Ở dạng GRU sử dụng trong PyTorch, trạng thái ứng viên được tính bởi:

$$\widetilde{h}_t = \tanh\left(W_n x_t + b_{in} + r_t \odot (U_n h_{t-1} + b_{hn})\right) \tag{5.14}$$

Trạng thái ẩn được cập nhật:

$$h_t = (1 - u_t) \odot \widetilde{h}_t + u_t \odot h_{t-1} \tag{5.15}$$

Nhờ Update Gate, GRU có thể quyết định mức độ giữ lại thông tin từ các bước trước và mức độ sử dụng trạng thái ứng viên tại bước hiện tại.
Việc sử dụng ít cổng hơn giúp GRU có số tham số thấp hơn LSTM khi hai mô hình có cùng input size và hidden size.

5.4.2 Kiến trúc GRU Classifier

Mô hình GRU được xây dựng với cấu hình tương tự LSTM:

- Input size: 28.
- Hidden size: 128.
- Số lớp GRU: 1.
- Sequence length: 28.
- Số lớp phân loại: 10.

Tại mỗi bước thời gian, GRU cập nhật trạng thái ẩn có 128 chiều.
Sau khi xử lý toàn bộ 28 hàng, trạng thái cuối cùng được sử dụng làm biểu diễn của ảnh:

$$h_{28} \in \mathbb{R}^{128} \tag{5.16}$$

Lớp Linear ánh xạ trạng thái này thành 10 logits phục vụ phân loại.
Luồng xử lý tổng quát:

$$\text{Image} \rightarrow \text{Row Sequence} \rightarrow \text{GRU} \rightarrow \text{Final Hidden State} \rightarrow \text{Linear} \rightarrow \text{Logits}$$

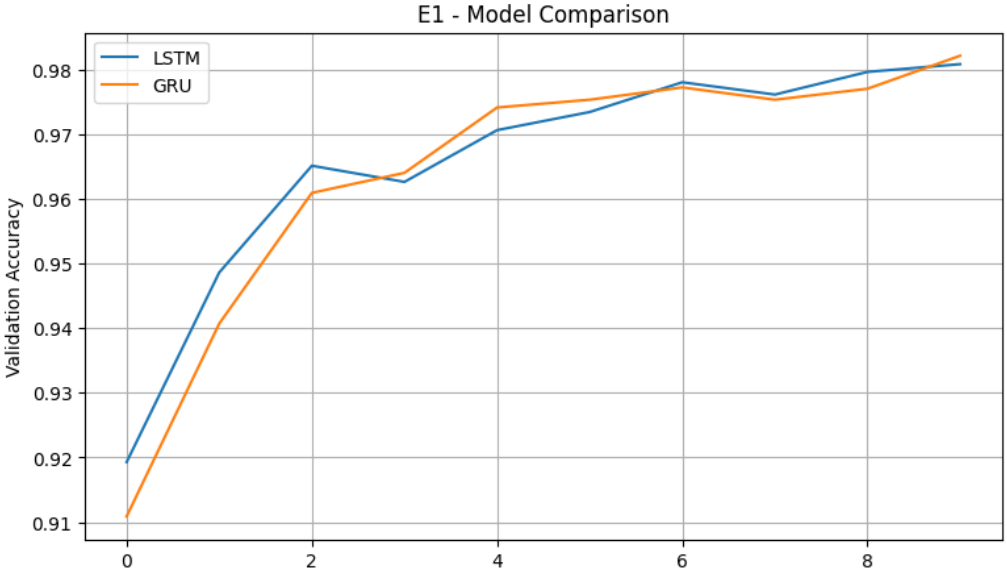
Tổng số tham số của mô hình là 61.962, thấp hơn LSTM do sự khác biệt trong cấu trúc các cổng điều khiển.

5.5 Kết quả thực nghiệm

Hai mô hình được huấn luyện với cùng tập dữ liệu, phương pháp Row Sequence và cấu hình cơ bản đã trình bày trong Chương 2.
Bảng 5.1 tổng hợp accuracy trên tập test, số tham số và thời gian huấn luyện.
Hình 5.1 thể hiện validation accuracy của hai mô hình trong 10 epoch huấn luyện.

Bảng 5.1: *Kết quả thực nghiệm E3*

Mô hình	Accuracy (%)	Parameters	Time (s)
LSTM	98,17	82 186	76,79
GRU	98,37	61 962	70,72



Hình 5.1: *So sánh validation accuracy của LSTM và GRU qua các epoch*

5.5.1 Nhận xét kết quả

Quan sát Hình 5.1, cả LSTM và GRU đều cải thiện độ chính xác nhanh trong những epoch đầu tiên và dần ổn định ở các epoch tiếp theo.

LSTM có xu hướng học nhanh hơn ở một số epoch đầu, trong khi GRU tiếp tục cải thiện và đạt kết quả tương đương hoặc cao hơn ở giai đoạn cuối của quá trình huấn luyện.

Kết quả test cho thấy LSTM đạt accuracy 98,17%, trong khi GRU đạt 98,37%, cao hơn 0,20 điểm phần trăm.

Về số tham số, GRU sử dụng 61.962 tham số, ít hơn 20.224 tham số so với LSTM, tương đương giảm khoảng 24,6%.

Thời gian huấn luyện của GRU là 70,72 giây, thấp hơn 6,07 giây so với LSTM.

Như vậy, trong cấu hình thực nghiệm này, GRU đạt accuracy nhỉnh hơn đồng thời có số tham số và thời gian huấn luyện thấp hơn.

Tuy nhiên, chênh lệch accuracy tương đối nhỏ và chỉ được ghi nhận trong một lần chạy. Do đó, chưa thể kết luận GRU luôn có hiệu quả tốt hơn LSTM.

5.6 So sánh với các mô hình E1

Để đánh giá hiệu quả của cách tiếp cận xử lý ảnh theo chuỗi, kết quả LSTM và GRU được đối chiếu với MLP và CNN trong E1.

Kết quả cho thấy cả LSTM và GRU đều đạt accuracy cao hơn MLP.

Cụ thể, LSTM cao hơn MLP 0,93 điểm phần trăm, còn GRU cao hơn 1,13 điểm phần trăm.

Điều này cho thấy việc xử lý ảnh theo chuỗi vẫn có thể tạo ra biểu diễn hữu ích để phân loại chữ số viết tay.

Tuy nhiên, CNN tiếp tục đạt accuracy cao nhất với 98,90%, cao hơn LSTM 0,73 điểm phần trăm và GRU 0,53 điểm phần trăm.

Bảng 5.2: *So sánh LSTM và GRU với baseline E1*

Mô hình	Accuracy (%)	Parameters	Time (s)
MLP	97,24	109 386	63,67
CNN	98,90	214 538	70,94
LSTM	98,17	82 186	76,79
GRU	98,37	61 962	70,72

CNN được thiết kế để khai thác cấu trúc không gian hai chiều, trong khi LSTM và GRU tiếp nhận ảnh theo thứ tự các hàng.

Sự khác biệt về cách xử lý dữ liệu có thể góp phần giải thích ưu thế của CNN đối với bài toán phân loại ảnh MNIST.

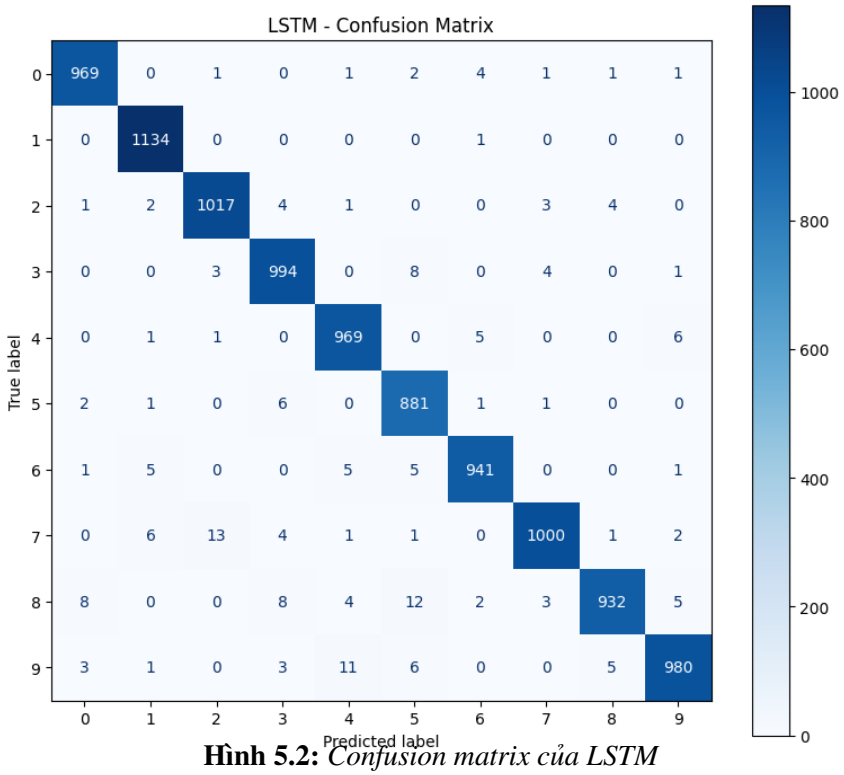
Xét về số tham số, LSTM và GRU đều có quy mô nhỏ hơn CNN. Đặc biệt, GRU có ít tham số nhất trong bốn mô hình được so sánh nhưng vẫn đạt accuracy trên 98%.

Về thời gian huấn luyện, các mô hình có kết quả khá gần nhau. Tuy nhiên, số tham số không phải yếu tố duy nhất quyết định tốc độ, do đặc điểm tính toán tuần tự của mạng hồi quy khác với CNN và MLP.

5.7 Phân tích lỗi và thảo luận

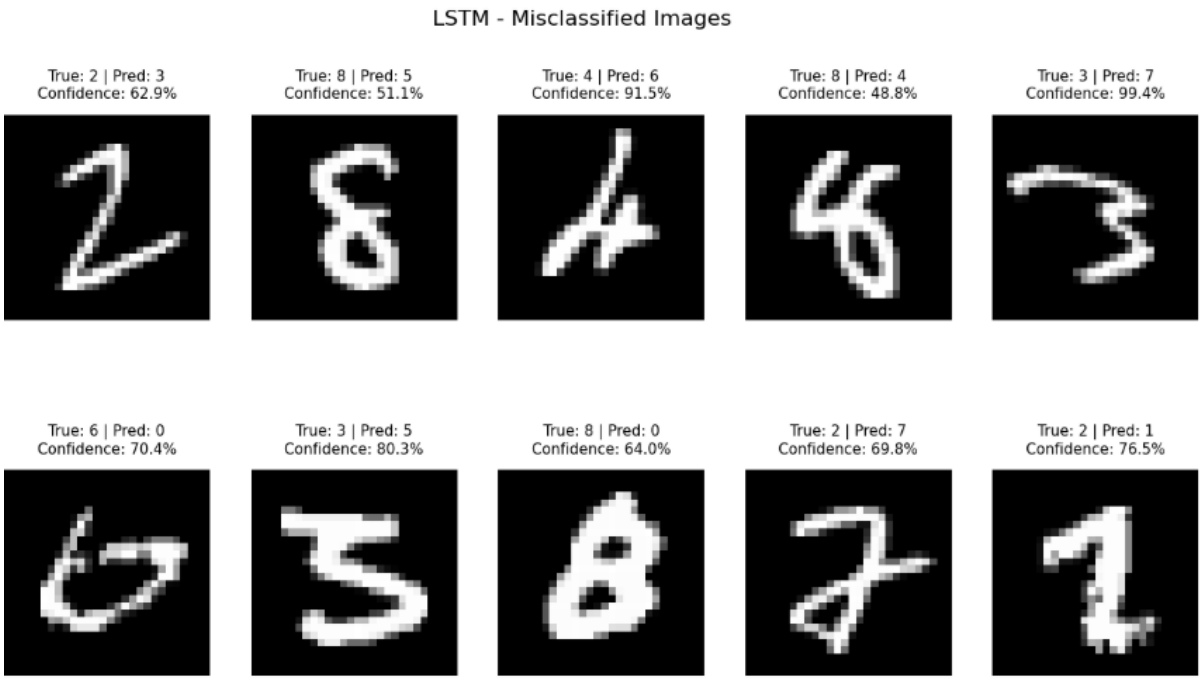
Để phân tích khả năng phân loại của hai mô hình, confusion matrix và các trường hợp dự đoán sai được sử dụng để đánh giá mức độ nhầm lẫn giữa các chữ số.

5.7.1 LSTM Classifier



Hình 5.2: *Confusion matrix của LSTM*

Hình 5.2 cho thấy phần lớn dự đoán của LSTM tập trung trên đường chéo chính, phản ánh khả năng nhận diện tốt các chữ số trong tập kiểm thử.



Total misclassified images: 183

Hình 5.3: *Một số trường hợp dự đoán sai của LSTM*

Tuy nhiên, vẫn xuất hiện những giá trị ngoài đường chéo, cho thấy mô hình còn nhầm lẫn giữa một số lớp chữ số.

Hình 5.3 minh họa những trường hợp dự đoán sai của LSTM. Các mẫu này có thể chứa những nét viết khó phân biệt hoặc hình dạng tương đồng với chữ số khác.

Do LSTM xử lý ảnh lần lượt từ hàng đầu đến hàng cuối, thông tin của các vùng ảnh được tổng hợp thông qua trạng thái ẩn theo thứ tự tuần tự.

Cách tiếp cận này có thể khai thác mối liên hệ giữa các hàng, nhưng không trực tiếp mô hình hóa quan hệ không gian hai chiều.

Tổng cộng, LSTM phân loại sai 183 trong 10.000 ảnh kiểm thử.

5.7.2 GRU Classifier

Confusion matrix của GRU trong Hình 5.4 cũng cho thấy các dự đoán tập trung chủ yếu trên đường chéo chính.

So với LSTM, GRU có tổng số trường hợp nhầm lẫn thấp hơn, phù hợp với accuracy cao hơn trên tập test.

Các trường hợp dự đoán sai trong Hình 5.5 cho thấy GRU vẫn gặp khó khăn đối với một số mẫu chữ số có nét viết không rõ ràng hoặc dễ bị nhận diện thành một lớp khác.

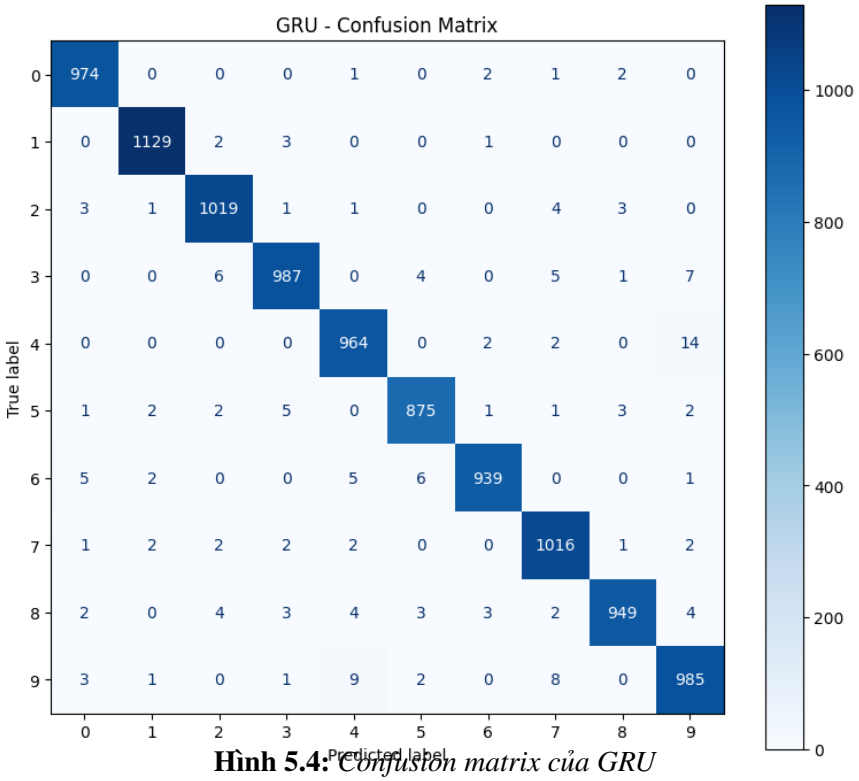
Mặc dù có cấu trúc cổng điều khiển đơn giản hơn, GRU vẫn duy trì khả năng tích lũy thông tin từ các bước thời gian trước đó để phục vụ phân loại.

Tổng cộng, GRU phân loại sai 163 ảnh, ít hơn LSTM 20 ảnh.

5.7.3 Thảo luận chung

Phân tích bốn hình cho thấy cả LSTM và GRU đều có khả năng phân loại hiệu quả khi ảnh được biểu diễn thành chuỗi các hàng pixel.

Hai confusion matrix có xu hướng tập trung mạnh trên đường chéo chính, trong khi những mẫu dự đoán sai cho thấy vẫn tồn tại một số trường hợp khó phân biệt.



Hình 5.4: *Confusion matrix của GRU*



Total misclassified images: 162

Hình 5.5: *Một số trường hợp dự đoán sai của GRU*

GRU có số lỗi thấp hơn LSTM, đồng thời sử dụng ít tham số hơn.

Tuy nhiên, sự khác biệt về kết quả không đủ lớn để khẳng định một kiến trúc luôn vượt trội so với kiến trúc còn lại.

Khi so sánh với E1, CNN đạt accuracy cao hơn cả LSTM và GRU. Điều này cho thấy khả năng khai thác đặc trưng không gian vẫn là một lợi thế quan trọng trong bài toán phân loại ảnh.

Trong khi đó, LSTM và GRU chứng minh rằng dữ liệu ảnh cũng có thể được xử lý hiệu quả dưới dạng chuỗi mà không nhất thiết phải sử dụng các phép tích chập.

5.8 Kết luận E3

Thực nghiệm E3 cho thấy LSTM và GRU đều có thể phân loại ảnh MNIST với độ chính xác cao khi sử dụng phương pháp Row Sequence.

LSTM sử dụng trạng thái ô nhớ và nhiều cổng điều khiển để tích lũy thông tin tuần tự, trong khi GRU sử dụng cấu trúc đơn giản hơn với hai cổng chính.

Trong kết quả thực nghiệm, GRU đạt accuracy 98,37%, cao hơn LSTM 98,17%, đồng thời có số tham số và thời gian huấn luyện thấp hơn.

So với baseline E1, cả hai mô hình đều vượt MLP về accuracy nhưng chưa đạt kết quả của CNN.

Kết quả cho thấy phương pháp biểu diễn dữ liệu và đặc điểm kiến trúc có ảnh hưởng đáng kể đến hiệu quả phân loại ảnh.

Chương 6

Kết luận

Thông qua ba bài tập E1, E2 và E3, báo cáo đã triển khai và đánh giá nhiều kiến trúc học sâu khác nhau trên bài toán phân loại ảnh MNIST, từ các mô hình cơ bản như Softmax, MLP và CNN đến Transformer, LSTM và GRU. Các thực nghiệm tập trung phân tích ảnh hưởng của kiến trúc và phương pháp biểu diễn dữ liệu đến độ chính xác phân loại, số lượng tham số và thời gian huấn luyện.

Trong E1, CNN đạt độ chính xác cao nhất với 98,90%, cho thấy hiệu quả của việc khai thác đặc trưng không gian trong dữ liệu ảnh. Ở E2, Transformer sử dụng CNN Embedding đạt 98,65%, vượt qua các phương pháp Patch và Row Embedding, tuy nhiên cũng yêu cầu nhiều tham số và thời gian huấn luyện hơn. Hai phiên bản MSA tự xây dựng và PyTorch cho kết quả gần tương đương khi sử dụng cùng cấu hình. Trong E3, GRU đạt 98,37%, cao hơn một chút so với LSTM (98,17%), đồng thời có số tham số và thời gian huấn luyện thấp hơn.

Nhìn chung, kết quả cho thấy không có một kiến trúc duy nhất tối ưu trên mọi tiêu chí. CNN đạt độ chính xác cao nhất trong các thí nghiệm, trong khi những mô hình khác thể hiện các ưu điểm riêng về cách biểu diễn dữ liệu và độ phức tạp. Điều này nhấn mạnh tầm quan trọng của việc lựa chọn kiến trúc phù hợp với đặc điểm bài toán và tài nguyên tính toán.

Tuy nhiên, các thí nghiệm mới chỉ được thực hiện trên MNIST với số epoch giới hạn và chưa đánh giá qua nhiều lần chạy với các seed khác nhau. Trong tương lai, có thể mở rộng nghiên cứu trên những tập dữ liệu phức tạp hơn, đồng thời thực hiện thêm các thí nghiệm để đánh giá tính ổn định và khả năng tổng quát hóa của từng kiến trúc.

Toàn bộ mã nguồn triển khai, cấu hình huấn luyện và kết quả thực nghiệm của ba bài tập E1, E2 và E3 được lưu trữ tại GitHub: `deep_learning_assignments`.